Deep Reinforcement Learning for Online Assortment Customization: A Data-Driven Approach

Tao Li
Center of Intelligent Decision-making and Machine Learning, School of Management, Xi'an Jiaotong University, littt1024@gmail.com

Chenhao Wang
School of Data Science, Chinese University of Hong Kong (Shenzhen), liziwch0111@gmail.com

Yao Wang
Center of Intelligent Decision-making and Machine Learning, School of Management, Xi'an Jiaotong University, yao.s.wang@gmail.com

Shaojie Tang
Naveen Jindal School of Management, The University of Texas at Dallas, Shaojie.Tang@utdallas.edu

Ningyuan Chen
Rotman School of Management, University of Toronto, ningyuan.chen@utoronto.ca

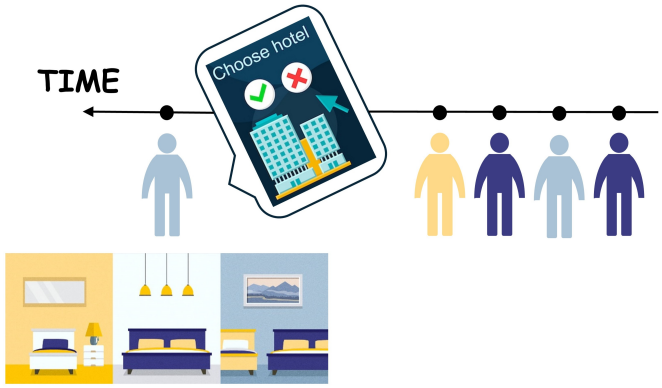
Abstract: When a platform has limited inventory, it is important to have a variety of products available for each customer while managing the remaining stock. To maximize revenue over the long term, the assortment policy needs to take into account the complex purchasing behavior of customers whose arrival orders and preferences may be unknown. We propose a data-driven approach for dynamic assortment planning that utilizes historical customer arrivals and transaction data. To address the challenge of online assortment customization, we use a Markov Decision Process (MDP) framework and employ a model-free Deep Reinforcement Learning (DRL) approach to solve the online assortment policy because of the computational challenge. Our method uses a specially designed deep neural network (DNN) model to create assortments while observing the inventory constraints, and an Advantage Actor-Critic (A2C) algorithm to update the parameters of the DNN model, with the help of a simulator built from the historical transaction data. To evaluate the effectiveness of our approach, we conduct simulations using both a synthetic data set generated with a pre-determined customer type distribution and ground-truth choice model, as well as a real-world data set. Our extensive experiments demonstrate that our approach produces significantly higher long-term revenue compared to some existing methods and remains robust under various practical conditions. We also demonstrate that our approach can be easily adapted to a more general problem that includes reusable products, where customers might return purchased items. In this setting, we find that our approach performs well under various usage time distributions.

Key words: online assortment, customization, deep reinforcement learning, simulation, reusable products

1 Introduction

In online assortment customization, the platform develops a policy to offer products to diverse customers who arrive over time with the aim of maximizing total profits over a finite-time horizon (Bernstein et al. 2015, Golrezaei et al. 2014, Chen et al. 2020). Many online retail scenarios, including hotel bookings, show-ticket sales, and the sale of short-life-cycle products, involve presenting customers with various assortments

Figure 1Online assortment customization with hotel booking as an example



from a limited inventory within a finite selling period. Our problem setting can be illustrated using the example of hotel booking, as shown in Figure 1. Different customers, represented by different colors, arrive one by one over time. When a customer arrives, the type is immediately revealed to the platform. The platform's objective is to provide an assortment with a cardinality constraint, and the customer can choose to purchase from the offered set or leave without buying anything. If a hotel is chosen, the platform receives income from the sale and reduces the inventory count by one. If a customer chooses not to purchase, the platform earns zero revenue. During the selling horizon, the platform cannot display sold-out products, which is a standard requirement in retailing applications (Gallego et al. 2015). The objective of the platform is to maximize total revenue over the selling horizon by generating assortments sequentially.

The input of our problem consists of historical arrival sequences and transaction data. However, the future customer arrival orders are unknown. To address this online optimization problem, assuming an adversarial pattern can be overly conservative and disregard historical arrival data. Conversely, fitting a stationary arrival pattern from data—assuming that different customer types arrive independently over time—can result in a misspecified assumption, as customer arrivals are typically random and follow a non-stationary pattern over time (Deng et al. 2022). The challenge of effectively utilizing historical arrival data in the online assortment optimization problem remains unresolved. The second challenge is to predict future customer choice behavior based on historical data. While researchers have put forth a variety of discrete choice models for such predictions, not all of them enable a tractable assortment optimization problem. Previous works on the online assortment customization problem usually assume a specific class of parametric choice models (Bernstein et al. 2015, Golrezaei et al. 2014, Rusmevichientong et al. 2020, Gong et al. 2022). However, the platform might find it challenging to select the appropriate parametric choice model and may face computational difficulty in enhancing assortment policy using nonparametric choice models. The final challenge to tackle concerns the limited inventory and the observation that assortment policies evolve over the selling period. Previous research has demonstrated that straightforward reservation strategies can enhance total

revenue given certain assumptions about the underlying choice model (Golrezaei et al. 2014). However, a more tailored assortment strategy aimed at maximizing long-term revenue remains to be developed.

Considering these challenges, we propose a novel Model-free Deep Reinforcement Learning (DRL) method to learn an assortment policy based on historical data. DRL, through its iterative interaction with the environment and progressive policy enhancement via trial and error, has proven to be an effective strategy for tackling real-world challenges associated with complex environmental dynamics in revenue management. Recent research in this area, including Oroojlooyjadid et al. (2022), Gijsbrechts et al. (2021), Yang et al. (2022), demonstrates the tremendous power of DRL in outperforming state-of-the-art heuristics. Using historical data, we construct a simulated environment by simulating customer arrivals and training a deep learning-based choice model, which exhibits high predictive accuracy, to simulate their choices. Subsequently, we interact with each customer by presenting assortments and observing their choices to iteratively enhance the assortment policy. Through our DRL agent's interaction with this simulated environment, we continuously pose the counterfactual question: what would the revenue outcome have been if we had implemented an alternative assortment policy instead of the documented one? By leveraging responses from the simulated environment, our DRL agent learns to refine the current policy, ultimately aiming to maximize the total expected revenue. Our contributions can be summarized as follows.

First, to the best of our knowledge, this is the first study that formulates the online assortment customization problem using deep reinforcement learning. Although DRL has been successful in various other applications, it is still challenging to apply to our problem, due to the unknown choice probabilities and the high-dimensional state and action spaces. This study not only provides a proof of concept that DRL, *when adapted properly using the problem structure*, can be a powerful tool for the online assortment customization problem, but also demonstrates the impressive performance against benchmarks in the literature.

Second, to approximate the optimal assortment policy, we develop *a special architecture* used in the deep neural network that combines *Recurrent Neural Network* (RNN) layers. Note that in general DNN is not designed to output a high-dimensional binary vector with various constraints. Our choice of the architecture allows the DNN to output an assortment under the resource constraints, in which RNN captures the complementary and substitutability of the products offered in the assortment and generates the included products sequentially, avoiding the curse of dimensionality. We believe the design of DNN architecture incorporating the problem features will open new research directions applying DRL to operations problems. We then use the DNN as the policy approximation in the Advantage Actor-Critic (A2C) algorithm to update the parameters.

Third, we demonstrate that our approach can readily incorporate reusable products. We add past sales outcomes into the state of our problem to capture the reusable nature. A time series filter is adopted to transform past sales results into the input of our model.

Finally, we compare our approach with several existing algorithms to validate its performance. Based on a fair ground-truth environment and a dataset generated from it, we show that our method is as good as or outperforms other benchmarks. The results under various settings demonstrate the robustness of our approach. To make our study reproducible and facilitate future research, we publicly release our code at `https://github.com/DRL-OM/DRL-assortment`.

1.1 Organization of the Paper

Section 2 provides a review of the related literature. Section 3 describes our problem setting in detail. Section 4 defines the MDP formulation of our problem and illustrates our model associated with its training algorithm. We present the numerical experiments in Section 5, including simulations based on a synthetically generated data set and a real-world transaction data set, to show the excellent performance and robustness of our approach. Section 6 describes how to adapt our model to consider reusable products, accompanied by extended experiments. Section 7 concludes our paper and provides several directions for future research. Most notations used in this work are listed in Table 1 (the subscript of t means at time period t).

Table 1 Symbol table

Notation	Meaning
T	Length of a selling horizon
N	Number of products
$\mathbf{r}$	Price vector
$\mathbf{I}_t$	Inventory level vector
$\mathcal{Z}$	Set of Customer types
z_t	Customer type
$\mathbf{S}_t$	State vector
$\mathbf{A}_t$	Available assortments under inventory and cardinality constraint
a_t	Assortment (action)
R_t	Reward
$\mathbf{o_t}$	Purchasing outcome vector
$\mathbf{O_t^L}$	Purchasing outcomes of past L periods

2 Literature Review

2.1 Online Assortment Problem

Assortment optimization, which aims to choose the best assortment by solving a revenue maximization problem, is a well-studied topic in revenue management. Our work is closely related to several existing works about optimizing assortments with limited inventory for heterogeneous online arriving customers. Bernstein et al. (2015) consider a stochastic arrival order where the distribution of customer types is known, and the revenue maximization problem can be formulated as a dynamic program (DP). However, this formulation suffers from the notorious "curse of dimensionality" problem since we need high-dimensional

state variables to record the remaining inventory of each product. They propose heuristics called Sub_0 and Sub_t instead of solving the DP numerically. Besides, Golrezaei et al. (2014) consider the online assortment customization problem in an adversarial setting, and propose an Inventory-Balancing algorithm with 0.5 $(1-1/e)$ competitive ratio guarantee in the asymptotic setting. The insight is to discount the price of products at low inventory levels, so they may be preserved for the long run. Different from the above two streams of arrival assumptions, we assume an unknown IID distribution of customer types, which is a moderate assumption between stochastic and adversarial. It assumes that the online arrival orders follow a distribution over some unknown base patterns, which are provided in the existing data set. Under this setting, Karande et al. (2011) and Alomrani et al. (2021) consider a well-known online bipartite matching problem. They claim that the arrival orders of online nodes are sampled from a base graph which can be revealed from an existing data set. However, our problem is more general than theirs in the way that we choose a subset of offline nodes one time and the matching between the offline nodes and the arriving online node is stochastic (the matching could be unsuccessful).

There are also several works relating to our extension where products are reusable. Under stochastic customer arrivals, the extra requirement of recording products that are currently in use makes the DP formulation even more intractable (Rusmevichientong et al. 2020). Rusmevichientong et al. (2020) propose a 1/2-approximation algorithm based on approximate dynamic programming. under the adversarial setting, the best algorithmic result is that the myopic policy is 1/2 competitive against the offline clairvoyant when the usage time of a product only depends on this particular product (Gong et al. 2022). Besides, Feng et al. (2019) prove that the aforementioned Inventory-Balancing algorithm is $(1-1/e)^2 \approx 0.4$-competitive for the case of large capacity. Under the same large capacity assumption, Goyal et al. (2020) propose a fluid-guided algorithm that achieves a guarantee of $(1-1/e)$. Differently, our approach doesn't depend on any assumption about product usage time or size of inventory. We record past sales outcomes to capture the reusable nature and transform them by a time series filter to input into our model.

The aforementioned works on online assortment customization are a special case of choice-based revenue management. A significant issue in this line of work is how to describe the choice probability of an alternative when faced with a specific assortment, and this is often captured by a specific choice model in the literature. Under the random utility maximization (RUM) principle, several choice models are proposed, including multinomial logit choice model (MNL) (McFadden et al. 1973), nested logit choice model (Talluri et al. 2004) and Markov chain choice model (MCCM) (Blanchet et al. 2016). Although they all have explanations for customer behaviors, they cannot capture irrational choices. Non-parametric models like rank-list model (Farias et al. 2009) and tree-based choice model (Chen and Mišić 2022) are proposed on this account. Furthermore, focusing on purely improving empirical predictive performance, several papers bridge the neural network with choice modelling by formulating choice modelling as a multi-class classification task and modelling it using neural networks. Bentz and Merunka (2000) start this line of work. They

propose a partially connected neural network with shared weights and softmax outputs for predicting choice probabilities, and this tends to possess stronger predictive power than the widely used MNL choice model in their experiments. In recent years, a number of works follow up, e.g., Han et al. (2022), Aouad and Désir (2022), Gabel and Timoshenko (2022), and Cai et al. (2022), extending the previous work to deep learning with different architectures in different application scenarios. Specifically, Gabel and Timoshenko (2022) propose a scalable deep-learning model in the coupon distribution context, and Cai et al. (2022) propose both feature-based and feature-free deep learning-based choice models. Although these neural choice models benefit from strong predictive performance, optimizing assortment based on them is hard due to their complex structure. Different from the literature on choice-based revenue management that assumes a specific choice model or a class of choice models, our approach is model-free. We fill the gap in choice-based revenue management literature to learn an assortment policy based on an arbitrary choice model. In offline simulations, we show that the policy learned by our approach through interacting with the simulated deep learning-based choice model beats the existing benchmarks. By learning a policy from a simulated environment built upon historical data, Our work provides a new data-driven approach in revenue management (Chen and Hu 2023).

2.2 Deep Reinforcement Learning

Reinforcement Learning (RL) defines the process where a goal-driven decision-maker interacts with the environment sequentially (Sutton and Barto 2018). Traditional RL methods like Q-learning (Watkins and Dayan 1992) are adopted for problems where the environment can be simply modelled (Rana and Oliveira 2014). In highly complex environments, deep neural networks are needed to act as function approximators, leading to deep reinforcement learning (DRL) methods. DRL has made a huge success in recent years in the fields like games (Mnih et al. 2015) and traffic planning (Xie et al. 2023).

Thanks to cross-disciplinary research, many innovative technologies have been applied in the field of OR/OM (Choi et al. 2022). This includes the application of RL in classical OR/OM problems. By modelling these problems as Markov Decision Processes (MDPs), RL leads to long-term reward maximization. For MDP with large state space, tractable solutions are infeasible, and here DRL can be applied to learn a near-optimal optimal policy. It has been shown that the performance of DRL beats conventional heuristics in many OR/OM applications. Bello et al. (2016) and Delarue et al. (2020) apply DRL in traditional combinatorial optimization problems, e.g. the travelling salesman problem, KnapSack problem, and vehicle routing problem. More recently, Kokkodis and Ipeirotis (2021) adopt DRL in online labour markets; Green and Plunkett (2022) train a DRL agent to bargain optimally for both sides as platforms and buyers on eBay; Alomrani et al. (2021) leverage DRL for solving the online bipartite matching problem, and the DRL agent can be trained to choose one conjoint offline node to match once an online node arrives, with the goal of maximizing the total number of matches. In the field of revenue management (RM), Yang et al. (2022)

leverage DRL to learn how to price and disclose the true information of fresh produce based on its current inventory and the remaining selling time; Liu (2022) develop DRL to learn a coupon targeting strategy; Oroojlooyjadid et al. (2022) and Gijsbrechts et al. (2021) train DRL agents for different inventory management tasks. Differently from the above problems in RM, the assortment problem is oriented to multiple products, and the substitution effects between products should be captured. While the actions in the above OR/OM applications are either to choose a particular node or to determine a specific value, the action in our problem is to choose a subset of products to offer once an online customer comes, which is more difficult because of its combinatorial nature.

DRL has also been adopted in the recommender system (Ie et al. 2019, Afsar et al. 2022). Our work is different from these in both the problem and the methodology. These works aim at improving the lifetime experience of a particular customer in the system. Differently, we focus on improving the assortment planning in a certain time period for several heterogeneous customers, with the existence of some realistic constraints. Specifically, Ie et al. (2019) develop a SlateQ algorithm based on DQN to maximize the long-term value of a recommendation set, similar to an assortment, for a user life cycle in a recommendation system. Instead of learning a Q-value for each assortment and selecting the highest Q-value assortment, they learn a Q-value for each individual product and solve a static assortment optimization problem, where the Q-values are treated as the prices of each product. However, this optimization problem is only tractable under specific choice models such as the MNL choice model, benefiting from the algorithm proposed in Rusmevichientong et al. (2010). With more accurate and complex choice models, the DQN-based approach is intractable, because the corresponding optimization problems are NP-hard. Moreover, realistic constraints like the inventory constraint and the cardinality constraint are not considered in Ie et al. (2019). We compare our method with other popular ones in the literature in the following Table 2.

Table 2 Comparison of our method with the ones in the literature

	consider limited inventory	utilize historical arrival data	arbitrary choice model
Myopic (Gong et al. 2022)	✗	✗	✗
Golrezaei et al. (2014)	✓	✗	✗
Bernstein et al. (2015)	✓	✓	✗
Rusmevichientong et al. (2020)	✓	✓	✗
SlateQ (Ie et al. 2019)	✓	✗	✗
Ours	✓	✓	✓

3 Problem Setting

We consider an online assortment optimization problem in a finite selling horizon. Each horizon consists of T periods, with one customer coming at each period. The platform holds a set of products $\mathcal{N} = \{1, 2, \ldots, N\}$ The initial inventory levels is denoted as $\mathbf{I}_0 = (I_{01}, I_{02}, \ldots, I_{0N})$, where I_{0i} represents the initial

inventory of product i. The revenue of product i is r_i, which is fixed in the selling horizon. There is no inventory replenishment during the selling horizon.

Let $\mathcal{Z} = \{1, \ldots, Z\}$ denote the set of customer segments. For the customer who arrived on time t, the type $z_t \in \mathcal{Z}$ is immediately revealed. Based on the type of customer, the remaining inventory and the selling horizon, the platform provides a feasible assortment a_t from the set of available assortments $\mathbf{A}_t = \{a | I_{ti} > 0, \forall i \in a; |a| < C\}$, where I_{ti} denotes the inventory level of product i at time t and C is the cardinality constraint of an assortment. The choice behavior of customer with the type z_t is represented by $p_{z_t}(i, a_t)$ for product $i \in a_t$.

We assume that the customer purchases at most one product and the no-purchase option is denoted as product 0. Therefore, $p_{z_t}(i, a_t) = 0, \forall i \notin a_t$, and $\sum_{i \in a_t} p_{z_t}(i, a_t) + p_{z_t}(0, a_t) = 1$. In time period t, the expected revenue $f(a_t, z_t)$ associated with the assortment a_t and customer type z_t is represented as follows:

$$f(a_t, z_t) = \sum_{i \in a_t} r_i p_{z_t}(i, a_t).$$

An assortment policy is a mapping that maps the current information of the platform, which consists of the current time period, the current inventory levels and the current arriving customer type, to the assortment to pick. The platform's goal is to develop an assortment policy π to maximize the total expected revenue:

$$\max_{a_t \sim \pi} \mathbb{E}_{z_1, \ldots, z_T} [\sum_{t=1}^{T} f(a_t, z_t)], \tag{1}$$

where the expectation is taken over the arrival order of the underlying customer types and the assortment policy (if stochastic).

The difficulty of this problem arises in complex choice behavior. For example, it is hard to formulate this problem explicitly under non-parametric choice models. Moreover, the total selling length is usually not fixed in reality, the optimal policy needs to consider the expected revenue, the left inventory, and the selling length. To this end, we have developed a data-driven approach that offers several nice properties. Firstly, we do not need to know the exact distribution of customer types like in the stochastic arrival assumption. We seek to learn a policy directly to maximize the total expected revenue when faced with the arriving orders of customer types in the historically recorded customer type arrival sequences. We claim that new arrival sequences originate from the same arrival pattern as the historical sequences so that the learned policy generalizes well when faced with new arrival sequences. Secondly, the choice probability of the underlying customers $p_{z_t}(i, a_t)$ can take an arbitrary form. A specific parametric choice model, such as the MNL choice model, results in a dynamic programming formulation of the problem (1) and corresponding heuristics based on the stochastic arrival order assumption (Bernstein et al. 2015). However, model misspecification can lead to significant revenue loss. In fact, there are many other, more complex choice models that align better with choice behaviors observed in historical transaction data than the widely used

MNL choice model, such as deep learning-based choice models (Aouad and Désir 2022, Cai et al. 2022). Nevertheless, these more complex choice models make the optimization problem (1) more challenging to analyze. Instead, our model-free approach does not rely on the exact formulation of problem (1). We begin by initializing a policy π and improve it through interaction with historically recorded sequences. Specifically, this interaction involves taking an action for each customer and receiving feedback, with the arrival order of customer types matching that of the recorded sequence. The feedback can be based on any choice model fitted from historical transaction data. This ensures that the choice model fits the historical data as accurately as possible. Consequently, we expect the learned policy to perform well when faced with new arrival sequences that exhibit choice behaviors similar to those in the historical data.

4 A2C for Online Assortment Customization

In this section, we describe our neural-network model and the reinforcement learning framework. Firstly, we formalize the online assortment customization problem as an MDP. Then we introduce our neural network model architecture. Lastly, we describe the Actor-Critic (A2C) training algorithm based on a simulator.

4.1 MDP Formulation

The online assortment customization problem can be formulated as a finite-horizon Markov Decision Process (MDP), represented by $\mathcal{M} = (T, \mathcal{S}, \mathcal{A}, R, P)$. We use T to represent the length of a selling horizon, i.e., the number of customer arrivals. $\mathcal{S}$ and $\mathcal{A}$ are the state and action space, respectively. At time period $t \in \mathcal{T}$, the platform agent observes a state $\mathbf{S}_t$ and needs to take an action a_t. A stochastic reward R_t is then revealed and the state transitions to another state. We detail each element of the MDP as follows:

- **State:** In time t, we observe the current inventory level $\mathbf{I_t} = (I_{t1}, \ldots, I_{tN})$ and the type of the arrivial customer z_t. Thus, the state vector $\mathbf{S}_t$ at time period t is expressed as:

$$\mathbf{S}_t = (\mathbf{I_t}, z_t, t).$$

- **Action:** Upon arriving a customer, the platform agent offers a feasible assortment. We use a N-dimensional binary vector $a_t = \{0,1\}^N$ to represent the action in time t, then the displayed assortment is $\{i \in \mathcal{N} | a_{ti} = 1\}$. Due to inventory and cardinality constraints, the action space is $A_t = \{a | I_{ti} \geq a_{ti}, \forall i \in \mathcal{N}; \sum_{i=1}^{N} a_{ti} < C\}$, indicating that recommendations cannot include products that are out of stock. This constraint leads to an exponentially growing number of possible actions based on the feasible products. Furthermore, actions across different time periods are interdependent due to the inventory constraint, meaning that a product cannot be recommended once it is out of stock.

- **Reward function:** The assortment optimization problem is revenue-driven, so we consider the revenue of the product sold in time period t as our reward R_t for that period:

$$R_t = \sum_{i=1}^{N} r_i (I_{ti} - I_{(t+1)i}).$$

This reward is a function of $\mathbf{S}_t, a_t, \mathbf{S}_{t+1}$. If the customer leaves without purchase, the reward is 0.

- **Environment dynamics:** With the elements introduced above, the MDP of our problem can be described as $(\mathbf{S}_1, a_1, R_1, \ldots, \mathbf{S}_T, a_T, R_T)$. The state $\mathbf{S}_t$ transitions to $\mathbf{S}_{t+1}$ after action a_t is taken at time period t. The stochastic transition is embodied in two aspects: The product inventory level I_{ti} changes if product i is sold in time period t; The customer type z_t changes if $\mathbf{z}_{t+1} \neq z_t$. Both these two incidents happen in a stochastic way:

$$\mathbb{P}(I_{(t+1)i} = I_{ti} - 1) = p_{z_t}(i, a_t), \mathbb{P}(I_{(t+1)i} = I_{ti}) = 1 - p_{z_t}(i, a_t),$$
$$\mathbf{z}_{t+1} \sim D(\mathcal{Z}),$$

where $D(\mathcal{Z})$ is the distribution of customer types. We use $P(\mathbf{S}_{t+1}, R_t | \mathbf{S}_t, a_t)$ to denote the joint distribution of $\mathbf{S}_{t+1}$ and R_t, when action a_t is taken at state $\mathbf{S}_t$:

$$P(\mathbf{S}_{t+1} = (\mathbf{I}_t, \mathbf{z}_{t+1}, t+1), R_t = 0 | \mathbf{S}_t, a_t) = (1 - p_{z_t}(i, a_t)) \times p(\mathbf{z}_{t+1}), \text{ for } i \in \mathcal{N},$$
$$P(\mathbf{S}_{t+1} = (\mathbf{I}_t^i, \mathbf{z}_{t+1}, t+1), R_t = r_i | \mathbf{S}_t, a_t) = p_{z_t}(i, a_t) \times p(\mathbf{z}_{t+1}), \text{ for } i \in \mathcal{N},$$
$$\sum_{\mathbf{z}_{t+1}} \sum_{i=1}^{N} [P(\mathbf{S}_{t+1} = (\mathbf{I}_t^i, \mathbf{z}_{t+1}, t+1), R_t = 0 | \mathbf{S}_t, a_t) + P(\mathbf{S}_{t+1} = (\mathbf{I}_t^i, \mathbf{z}_{t+1}, t+1), R_t = r_i | \mathbf{S}_t, a_t)] = 1,$$

where $\mathbf{I}_t^i \doteq (I_{t1}, \ldots, I_{ti} - 1, \ldots, I_{tN})$.

A Policy π in the MDP maps a state to a distribution of all possible actions, $\pi : \mathbf{S}_t \mapsto \mathbf{A}_t$. Following policy π, the future total return starting from time period t can be described as $G_t^\pi(\mathbf{S}_t)$:

$$G_t^\pi(\mathbf{S}_t) = \sum_{l=0}^{T-t} \mathbb{E}_\pi[R_{t+l} | a_{t+l} \sim \pi]$$

Define the optimal value function $v^*(\mathbf{S}_t) = \max_\pi G_t^\pi(\mathbf{S}_t)$, indicating the maximum expected return we can get starting from state $\mathbf{S}_t$. Assuming countable states and actions and known $P(\mathbf{S}_{t+1}, R_t | \mathbf{S}_t, a_t)$, the optimal value functions and optimal actions can be obtained by solving the well-known Bellman equations (Bellman 1954):

$$v^*(\mathbf{S}_t) = \max_{a_t \in A_t} \sum_{\mathbf{S}', r} P(\mathbf{S}', r | \mathbf{S}_t, a_t)[r + v^*(\mathbf{S}')].$$

However, note that the form and value of $p_{z_t}(i, a_t)$ and $D(Z)$ are unknown to the DRL agent, so the joint distribution $P(\mathbf{S}_{t+1}, R_t | \mathbf{S}_t, a_t)$ is unknown in advance. On the other hand, with a larger number of products N, large state space and particularly large action space make this equation intractable. Furthermore, when we consider reusable products, the requirement to record the in-use products leads to even higher-dimensional state variables, see the detail in Section 6.

To overcome the challenge that the arrival orders are unknown beforehand, we seek to learn an assortment policy straightforwardly from interactions with historical arrival sequences. The policy is learned to achieve the maximized total revenue faced with historical customer arrival sequences and is expected to perform

well when facing new sequences, assuming that the arrival pattern of new sequences and the underlying customer behavior are the same as in historical data. Note that in the interaction process, customers' feedback can be an arbitrary choice model fitted from historical transaction data, thus the nonparametric choice models with nice empirical performance can be adopted. To overcome the challenge of large action space, we leverage a Recurrent Neural Network (RNN) to specify our assortment policy π_{θ_p}, where θ_p denotes the parameter of RNN. We call it the policy network. We want to find a policy to maximize total reward in a selling horizon:

$$\theta_p^* = \underset{a_t \sim \pi_{\theta_p}, t=1,\ldots,T}{\arg\max} \ \mathbb{E}_{\mathbf{s}_1, a_1, \ldots, \mathbf{s}_T, a_T}[R_1 + \cdots + R_T].$$

4.2 Model Architecture

We develop a DNN model to generate assortment and train it by the A2C algorithm. There are three streams of the DRL method for policy learning. The value-based method, such as Deep Q-learning (Mnih et al. 2015), keeps track of state-action value function $Q^{\pi}(s,a)$ for every state-action pair. Given a state, one can choose the action that maximizes the value function. However, in our problem with combinatorial action space, the optimal Q-function is hard to learn, and the arg max search among all actions is intractable. On the other hand, Policy-based RL like Reinforce (Williams 1992) adopt a policy network to generate a distribution over all actions, and we can sample an action according to this distribution. The parameters of the policy network are updated by gradient descent. Although the learning process is simple, it is less stable and suffers from the high variance issue. At the junction of these two methods, the Actor-Critic Architecture builds an actor to guide the action, and a critic is developed to evaluate the performance of this action. We build up a model consisting of the policy network, the value network and the shared layers.

Figure 2Our DNN Architecture

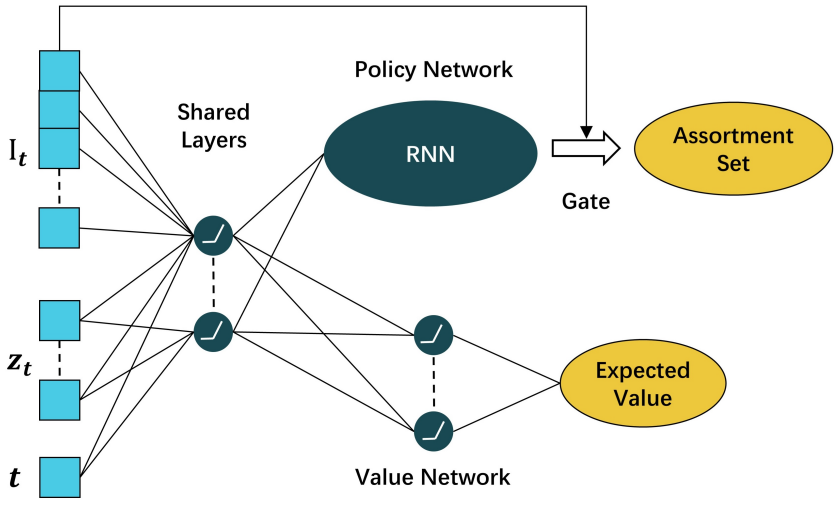


Figure 2 describes our model. The input of our model, which is the state vector, includes information about both products and the arriving customer. The state vector is put into a multi-layer fully connected neural network. This aims to model the preference of products among different customer segments, considering the complex relationships among products. The relationship includes the important substitution and complementary phenomenon, which is hard to capture by a parametric choice model. The output of these shared hidden layers, which is named the learned state, acts as the input of both the value network and the policy network. The adoption of shared layers is meant to benefit convergence. The value network is a fully connected neural network, which produces the estimated expected return starting from this learned state. The policy network is a recurrent neural network, which generates the assortment set in an auto-regressive way so that the chosen products are closely related. The detail about the generation process is shown in 4.2.1.

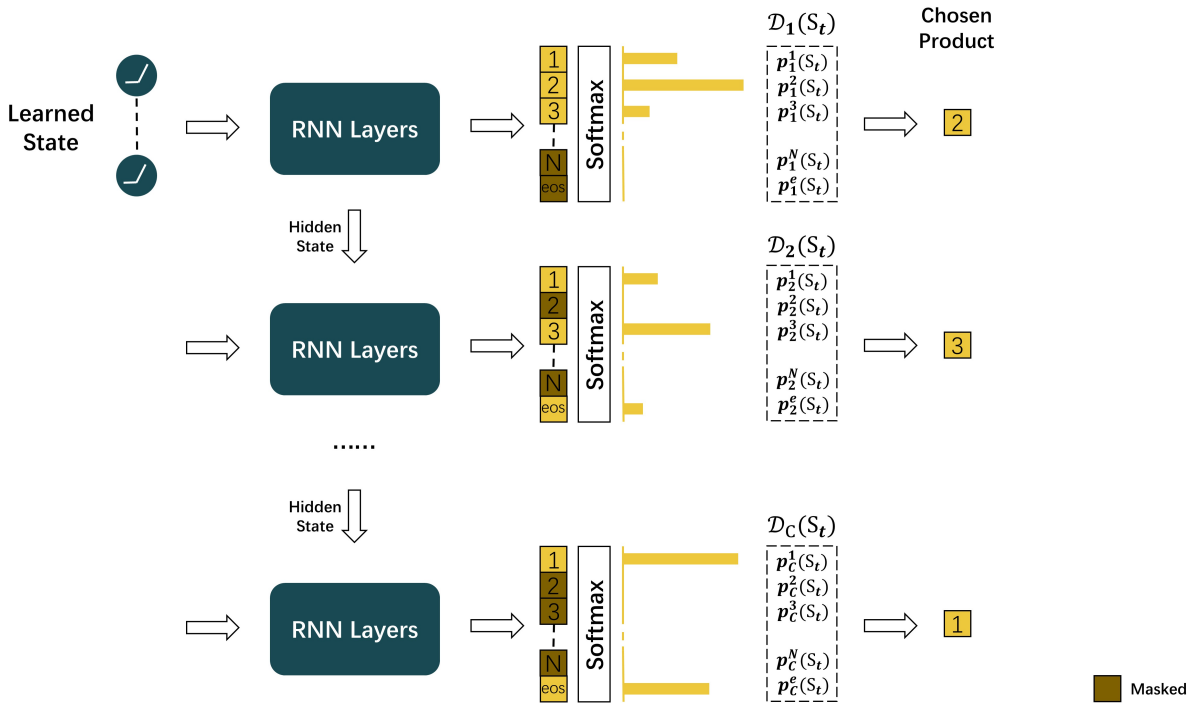
We use θ to denote all the parameters in the model that need to be fine-tuned through training, including the parameter set θ^s in the shared layers, the parameter set θ^p in the policy network and parameter θ^v in the value network. Given θ, Our model generates a value function $v^\pi(\mathbf{S}_t;\theta^s,\theta^v)$ and a policy $\pi(\mathbf{S}_t;\theta^s,\theta^p)$ for input state $\mathbf{S}_t$ in time period t. The adoption of the value function is aimed at stabilizing the learning process of the assortment policy.

4.2.1 Assortment Action

The policy network is the heart of our architecture. It generates products one by one in a sequence. Specifically, in order to offer products under cardinality constraint, the RNN maps the learned state to a sequence of C outputs, each corresponding to a $(N+1)$-dimensional normalized probability vector $\mathcal{D}_1(S_t),\mathcal{D}_2(S_t),\ldots,\mathcal{D}_C(S_t)$, as is shown in Figure 3. Each probability vector is expressed as $\mathcal{D}_k(S_t) = (p_k^1(S_t),p_k^2(S_t),\ldots,p_k^N(S_t),p_k^e(S_t)), k\in\{1,2,\ldots,C\}$, where the first N elements correspond to the probability of products $\{1,2,\ldots,N\}$ being chosen, and the last element corresponds to the probability of stopping the generation process, named the "end-of-sequence" ("eos"). The generation of optimal assortments benefits from the fact that the latter probability vectors relate to former probability vectors by hidden states produced from the RNN layers so that the relationships of products are considered in the generation process. we sample orderly according to these probability vectors to generate different products in at most C rounds. To ensure that products out of stock will never be picked into the assortment, we mask the products out of stock so that their chosen probabilities are 0 in the whole process. This is the so-called "gate" step. The "eos" token is masked in the first round. Besides, after one product is sampled we mask it in the later generating process. Until C products are chosen or the "eos" token is sampled, this generating process stops. We get a sequence of c products $h_1,h_2,\ldots,h_c$ constituting the final assortment set a_t, where $c\le C$. This stochastic policy can be formulated as:

$$\pi_\theta(a_t=\{h_1,h_2,\ldots,h_c\}\mid\mathbf{S}_t)=\begin{cases}p_1^{h_1}(S_t)\times p_2^{h_2}(S_t)\times\cdots\times p_c^{h_c}(S_t) & \text{if } c=C,\\ p_1^{h_1}(S_t)\times p_2^{h_2}(S_t)\times\cdots\times p_c^{h_c}(S_t)\times p_{c+1}^{e}(S_t) & \text{if } c<C.\end{cases} \tag{2}$$

Figure 3Policy Network



Following Alomrani et al. (2021), while validation and testing, we adopt a deterministic policy that picks the highest probability in $\mathcal{D}_k(S_t), k \in \{1,2,\ldots,C\}$ to generate the assortment a_t. This specific RNN-based policy network not only helps generate assortments adhering to realistic constraints but also helps find the optimal assortment at each state, considering the relationship between products through the sequential generation process. We conduct ablation experiments to demonstrate its priority in E-Companion EC.8 by comparing it with a policy network constructed by a standard multilayer perceptron (MLP).

4.3 Training Algorithm

For our problem with huge action space, an optimal Q function is hard to construct and learn. Besides, policy-based RL algorithms suffer from the problem of high variance. So we adopt the A2C algorithm to update our model by interacting with the environment. A customer comes every time period and reacts according to the assortment offered by our model. T time periods construct a selling horizon. While interacting with each coming customer, the parameter θ is updated every M time period, through a training buffer of observed states, actions, and received rewards. M is a hyper-parameter that we should decide. During the k-th training buffer, $k \in \{1,2,\ldots,\lceil T/M \rceil\}$, we follow the current stochastic policy π_k for M subsequent periods starting from state $\mathbf{S}_{M(k-1)}$ (the length of the last buffer may be less than m if we come to the ending of the selling season). We keep track of the facing states $(\mathbf{S}_{M(k-1)},\ldots,\mathbf{S}_{Mk})$, the taken actions $(a_{M(k-1)},\ldots,a_{Mk})$, the incurred rewards $(R_{M(k-1)},\ldots,R_{Mk})$ and the resulting state $\mathbf{S}_{Mk+1}$ after this training

buffer. We can compute the loss L_k in this training buffer given the record, and then use an optimizer like Adam to update current parameters in the direction of gradient to reduce the total loss, where α is the step size:

$$\theta_{k+1} = \theta_k - \alpha \nabla_\theta L_k. \tag{3}$$

The loss L_k consists of three parts: value loss L_k^v, policy loss L_k^p, and entropy loss L_k^e. We detail them separately.

Value loss measures how well the value network estimates expected future reward at each time period. At time period l in k-th training buffer, $l \in \{M(k-1), \ldots, Mk\}$, the recorded state $\mathbf{S}_l$ corresponds to an expected future value estimation by current parameter: $v(\mathbf{S}_l; \theta^s, \theta^v)$. Based on the recorded rewards and the final resulting state, we can compute another value expectation: $R_l + \mathbb{I}(l \neq T)(\sum_{t=l}^{Mk}(R_t) + v(\mathbf{S}_{Mk+1}; \theta^s, \theta^v))$, where $\mathbb{I}(l \neq T)$ is the indicator function of whether it comes to the end of the selling season. Note that we don't consider a discount factor here. This value expectation is more realistic since it contains real feedback from the environment. We name the difference between these two value estimations as the Advantage at time period l:

$$Advantage_l = R_l + \mathbb{I}(l \neq T)(\sum_{t=l}^{Mk}(R_t) + v(\mathbf{S}_{Mk+1}; \theta^s, \theta^v)) - v(\mathbf{S}_l; \theta^s, \theta^v). \tag{4}$$

In order to get a more accurate value estimation, we want this difference to be as small as possible. Value loss of the k-th training buffer is the mean squared Advantage of M time periods where the Advantage is calculated by (4):

$$L_k^v = \sum_{t=M(k-1)}^{Mk} Advantage_t^2 / M. \tag{5}$$

Policy loss measures how well the generated action performs. For the "good" actions, we want to increase the probability of being chosen. The above-introduced Advantage can be a good critique of the performance of actions. At time period l, The advantage can be positive(means that we have taken a good action, and we should increase its probability), zero, or negative(means that the action taken is not optimal under the current critic network, and we should decrease its probability). For the set action a_l at time period l, the probability of it being taken, which is denoted as $\pi_\theta(a_t \mid \mathbf{S}_t)$, is calculated by (2). We define the policy loss as below:

$$L_k^p = -\sum_{t=M(k-1)}^{Mk} Advantage_t \cdot (\log \pi_\theta(a_t \mid \mathbf{S}_t)). \tag{6}$$

Entropy loss is used to ensure that we can explore more actions and avoid A2C converging to local minima. The entropy of time period l in k-th training buffer is defined as the sum of the entropy of each product i: $\sum_{i=1}^{N} p_i(\mathbf{S}_l) \log p_i(\mathbf{S}_l)$. This entropy is maximized when all products share the same probability of putting into the assortment at this time. So entropy loss is defined as negative total entropy of k-th training buffer:

$$L_k^e = -\sum_{l=M(k-1)}^{Mk} \sum_{i=1}^{N} p_i(\mathbf{S}_l) \log p_i(\mathbf{S}_l). \tag{7}$$

The total loss function can be expressed as below:

$$L_k = \beta_v L_k^v + \beta_p L_k^p + \beta_e L_k^e. \tag{8}$$

Where L_k^v, L_k^p, L_k^e are calculated as in (5), (6), (7), and $\beta_v, \beta_p, \beta_e$ are the regularization factors of value loss, policy loss, and entropy loss, respectively. The total loss L_k is used for parameter updating in (3).

The training process based on historical arrival sequences is detailed in Algorithm 1. The recorded historical arrival sequences $[H]$ reveal the arrival order of customer types in different selling horizons. For each sequence, the A2C agent interacts with customers sequentially with types recorded in this sequence. Specifically, the A2C agent offers an assortment based on the current parameter set for each customer and this customer makes his choice based on a fitted choice model ϕ. The feedback mechanism ϕ can be in any form, fitting the historical transaction records as precisely as possible. While training, we know the length of each sequence, and thus the termination of each horizon. In addition, the horizon terminates prematurely if all products run out of inventory.

Algorithm 1 A2C for Online Assortment with Cardinality Constraint

Require: Historical arrival sequences $[H]$, environment feedback ϕ, cardinality C, model update step M.

Initialize the shared layer θ^s, the policy network θ^p and the value network θ^v;
for $h \in \{1, \ldots, H\}$ **do**
  Initialize the inventory level I_{i0} of each product i;
  Customers arrive with their types recorded in h orderly;
  repeat
    Initialize the observational buffer $B = \{\}$;
    for $t = 1, \ldots, M$ **do**
      Observe the current arriving customer type, input state $\mathbf{S}_t$, sample action a_t according to (2);
      The arriving customer chooses from a_t according to ϕ, observe reward R_t;
      Append tuple $\{\mathbf{S}_t, P(i|\mathbf{S}_t), i \in \{1, \ldots, N\}, v(\mathbf{S}_t), R_t, \mathbb{I}(t \neq T)\}$ into observational buffer B ;
    end for
    Calculate the total loss function L_k based on observational buffer B, update parameters $\theta^s, \theta^p, \theta^v$;
    Decrease the learning rate α;
  until $t = T$ or $I_i = 0, \forall i \in \{1, \ldots, N\}$.
end for

5 Numerical Experiments

In this section, we verify the performance of the proposed DRL approach through semi-synthetic numerical experiments. In Section 5.1, we conduct simulations based on a synthetic data set built upon a pre-assumed customer type distribution and ground truth feedback. First, we introduce the construction of the environment and the generation of synthetic transaction data in 5.1.1. We train and test the performance of our algorithm in EC.3, followed by results shown in 5.1.3 and robustness checks shown in in EC.4. In Section 5.2, we conduct simulations based on a real-world hotel-booking data set. We introduce the data set and a simulator fitting the extracted data in EC.6, and then we test our algorithm in 5.2.1.

Below we introduce several existing approaches for the online assortment customization problem to compare with our approach.

-**SlateQ-MNL**: SlateQ-MNL is a DRL algorithm based on deep Q-learning (Ie et al. 2019) and the MNL choice model. A set of MNL parameters is first fitted and then a deep Q network (DQN) is trained. We adopt the same MDP formulation stated in 4.1 for training SlateQ-MNL, but the way the actions are taken in SlateQ-MNL is different from that in our method. Specifically, SlateQ-MNL learns the Q-value of each product at each state of the MDP, then we solve a revenue maximization problem with cardinality constraint based on the fitted MNL choice model, regarding the i-th Q-value $q_i(\mathbf{S}_t)$ as the price of product i at state $\mathbf{S}_t$:

$$a_t = \underset{a\in A_t,|a|\le C}{\arg\max} \sum_{i\in a} q_i(\mathbf{S}_t) p_{z_t}(i,a).$$

-**Random**: Random means that we randomly pick $1 \sim C$ products from the available product set into the assortment, regardless of the arriving customer type and remaining inventory level.

-**Myopic-MNL**: Based on the fitted MNL parameter, Myopic-MNL solves a revenue maximization problem with cardinality constraint based on the MNL choice model without considering inventory:

$$a_t = \underset{a\in A_t,|a|\le C}{\arg\max} \sum_{i\in a} r_i p_{z_t}(i,a).$$

-**EIB-MNL**: EIB-MNL is an online algorithm proposed by (Golrezaei et al. 2014). At each time period we solve the assortment optimization problem under cardinality constraint with the prices discounted by exponential penalty function $\Psi(x) = (e/(e-1))\,(1-e^{-x})$, where I_{0i} denotes the initial inventory level of product i and $I_{(t-1)i}$ denotes the inventory level of product i at the beginning of time period t:

$$a_t = \underset{a\in A_t,|a|\le C}{\arg\max} \sum_{i\in a} \Psi(I_{(t-1)i}/I_{0i}) r_i p_{z_t}(i,a).$$

-**Sub$_t$-MNL**: Sub$_t$-MNL (Bernstein et al. 2015) assumes the stochastic arrival pattern with a fixed selling length and a Poisson arrival process. An approximate dynamic program is solved for online assortments, considering the limited inventory.

-**DP-GR-MNL/DP-RO-MNL**: DP-GR-MNL and DP-RO-MNL are the greedy policy and the rollout policy proposed in Rusmevichientong et al. (2020), respectively. A dynamic program is formulated assuming the stochastic arrival pattern and linear approximations are constructed for the optimal value functions. DP-GR-MNL solves assortment optimization problems based on the approximated functions and offers a fixed assortment to a specific customer type at each time period. DP-RO-MNL further considers the inventory levels of the products at each time period.

The implementation details of Myopic, Sub_t, EIB, DP-GR and DP-RO policy are put in E-Companion EC.1. Following Golrezaei et al. (2014) and Rusmevichientong et al. (2020), we compare our method with these benchmarks based on the offline linear programming (LP) upper bound:

$$\begin{aligned}
\max & \sum_{t=1}^{T} \sum_{a\in\mathscr{A}} \sum_{i=1}^{n} r_i p_{z_t}(i,a) y^t(a) \\
\text{s.t.} & \sum_{t=1}^{T} \sum_{a\in\mathscr{A}} p_{z_t}(i,a) y^t(a) \le c_i \quad 1 \le i \le n \\
& \sum_{a\in\mathscr{A}} y^t(a) = 1 \quad 1 \le t \le T \\
& y^t(a) \ge 0 \quad 1 \le t \le T, a \in \mathscr{A}
\end{aligned}$$

This LP upper bound assumes that customer arrival order in a horizon is already known, so it is called offline. The overall feasible set of assortments can be expressed by $\mathscr{A} = \{a : |a| \le C\}$. The decision variables $y^t(a)$ indicate the probability that assortment a should be offered to the customer arriving at time t of type z_t. Note that in reality we can either offer an assortment ($y^t(a) = 1$) or don't offer it ($y^t(a) = 0$), while the decision variables $y^t(a)$ are fractional and thus more flexible. This means that the resulting upper bound can never be achieved. In the testing results below, the performance of our method and all benchmarks is expressed as the fraction of the average LP upper bound to see how optimal is each approach.

5.1 Simulation With Ground Truth feedback

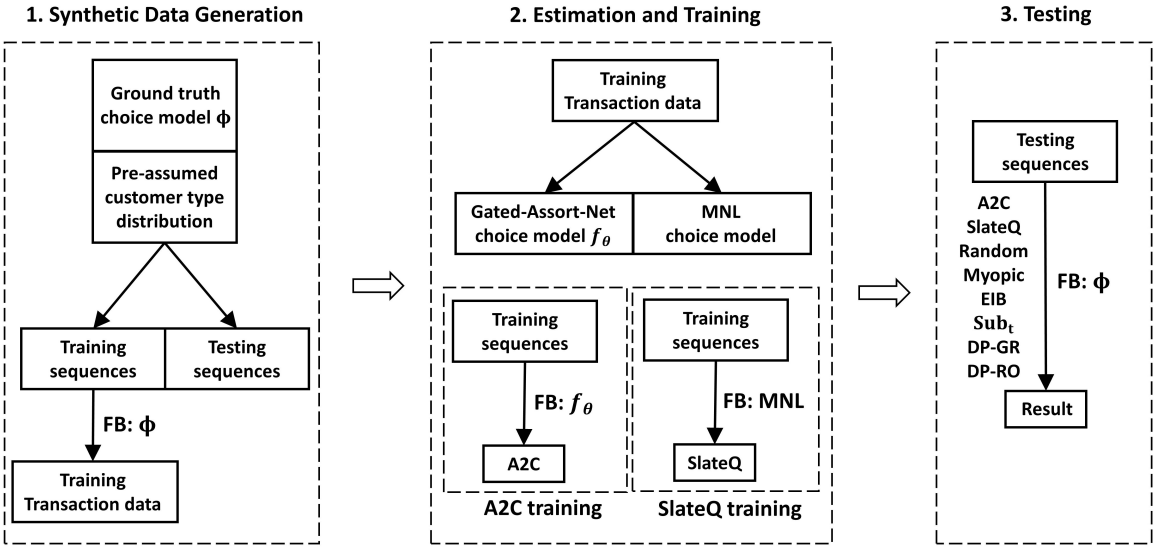
In this section, we construct an environment where the customers arrive according to a pre-assumed arrival pattern and give feedback to the facing assortments according to a realistic ground truth choice model. The overall simulation process is depicted in Figure 4, and we will detail it below.

5.1.1 Environment Construction and Transaction Data Generation

In our problems, we have ten products indexed by $\{1,2,\ldots,10\}$ and four customer types indexed by $\mathcal{Z} = \{1,2,3,4\}$. We sample a price $r_i \sim U[10,25]$ for each product and reorder these products according to their prices so that $r_1 \le r_2 \le \cdots \le r_{10}$. The initial inventory of each product is set at 12. We also try different initial inventory levels and larger cases with more products for robustness checks, as shown later in E-Companion EC.4.

Following Aouad et al. (2022), we construct a ranking-based choice model as environment feedback, which consists of several preference lists and a distribution over these lists, as interpreted by ϕ in Figure 4.

Figure 4Simulation flow chart



A preference list ranks the products from most to least preferred and the customer with this preference list will deterministically choose the highest-rank item that appeared in the assortment. The distribution introduces the randomness of customer choice behavior. We build each preference list by constructing two sub-lists as follows. In the first five lists, we combine a sub-list consisting of a permutation of the first two products and a sub-list consisting of a permutation of the last eight products, and insert a no-purchase option as product-0 into the second sub-list randomly. As a result, the first five lists represent a high preference for the two lowest-price products. We repeat this step four times so that we get 20 preference lists, which consist of four groups, each consisting of five lists, indicating a preference for products with different prices, ranking from lowest to highest. We add an extra 21st preference list with the no-purchase option ranking first in the list. We set 4 customer types in our environment, which are represented by 4 distinct distributions over the 21 preference lists. The probability for the 21st preference list is set as 0.1 in each distribution. We correspond the four customer types to the aforementioned four groups of preference lists as follows. For customer type 1, we generate its corresponding distribution by independently sampling five values $\beta_1, \ldots, \beta_5$ from an exponential distribution with mean 1, and set the distribution over the 20 preference lists as $\lambda = (0.9\beta_1/\sum_{l=1}^{5}\beta_l, \ldots, 0.9\beta_5/\sum_{l=1}^{5}\beta_l, 0, \ldots, 0)$. That is saying, customer type 1 has a high preference for the two lowest-price products. We set the distribution of the customer type 2 over the 20 preference lists as $\lambda = (0.9\beta_1/\sum_{l=1}^{10}\beta_l, \ldots, 0.9\beta_{10}/\sum_{l=1}^{10}\beta_l, 0, \ldots, 0)$, and customer type 3 and 4 follow a similar way. By constructing in such a way we let customer type 1 have the smallest consideration set while customer type 4 have the largest consideration set. Each customer type behaves stochastically according to the distribution over the 21 preference lists, which we refer to as the environment's ground truth feedback. This ground truth choice model is used for synthetic data generation and acts as feedback from the environment in the testing process.

In our problems, we set the customer arrival pattern similar to the way in Rusmevichientong et al. (2020), so that customer type 1 tends to arrive first and customer type 4 tends to arrive later, and we should protect inventory for those with a larger consideration set. Particularly, we set the number of customers T on each horizon sampled uniformly at random from $U(90,110)$. We then set four equally spaced time points $\tau^1 \leq \tau^2 \leq \ldots \leq \tau^n$ over the selling horizon. Let $p^{t,z}$ be the probability a customer of type z arrives at time point t, $p^{t,z} = e^{-\kappa|t-\tau^z|} / \sum_{z' \in \mathcal{Z}} e^{-\kappa|t-\tau^{z'}|}$, where κ is a parameter that we set as 0.03. So, the arrival probability for a customer of type z peaks at around time point τ^z. We set the four peaks as $20, 40, 60, 80$.

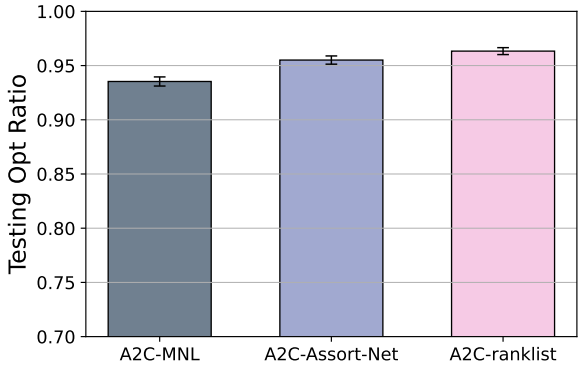
Following this arrival pattern, we generate a stream of synthetic arrival sequences for 500 horizons. We separate the first 400 sequences $\{(z_1, \ldots, z_{T_d})\}_{d=1}^{400}$, and use the remaining 100 sequences $\{(z_1, \ldots, z_{T_d})\}_{d=401}^{500}$ to act as possible arrival sequences in the future horizons for testing our approach and benchmarks. We further generate synthetic transaction data $\{(a_t, y_t)_{t=1}^{T_d}\}_{d=1}^{400}$ aside the training sequences. In particular, for each horizon and each arriving customer of type z in this horizon, we offer him 4 products randomly sampled from the 10 products (regardless of the inventory levels), namely an assortment a_t, and we receive the purchasing feedback (shown by "FB" in Figure 4) $y_t \in \{0, 1 \ldots, 10\}$ according to the ground truth choice model of the particular customer type. Thus, information about the ground truth choice model can be learned from the transaction data. We defer the statements about the training and testing process to E-Companion EC.3, and show the main testing result in the next section.

5.1.2 Impact of Simulator

First, we examine the impact of the simulator on the performance of our A2C method. Different simulators have different abilities to fit the historical transaction data, thus deviating differently from the ground truth choice model. As shown in (Cai et al. 2022), Gated-Assort-Net achieves better log-likelihood than structural choice models like the MNL choice model. However, it's still unclear whether the better empirical performance of the choice models can help our A2C learn a better assortment policy when adopting these choice models as simulators in the training process. Thus, we investigate this question by training three different A2C agents adopting three different choice models as simulators, respectively. The first two choice models are the MNL choice model and Gated-Assort-Net, both estimated from the synthetic transaction data. The last choice model is the rank list choice model with the same parameters as the ground truth that generates the transaction data.

The testing results of the learned three A2C agents are shown in Figure 5. Among the three simulators, we know that the MNL choice model fits the transaction data worst, while the rank list choice model fits the transaction data best. The testing results show that when testing with the ground truth feedback, the A2C agent performs better when trained with the simulator that fits the historical transaction data better, i.e., the A2C agent trained with the ground truth feedback performs best among these three. This is because the ground truth feedback is the same in the testing process as in the synthetic data generation process, and

Figure 5Testing results with three different simulators



a simulated environment that is closer to the true environment helps the A2C agent learn better how the customers behave and thus offer better assortments. This result indicates that while the true form of the underlying customer behavior is complex and unknown to us, which is the case in the real world, it's better to train our A2C agent with a simulator that fits the historical transaction data better. The fact that our A2C agent can be trained with any form of simulator is one of the special strengths of our approach, compared with existing methods that rely on specific structural choice models to make the assortment optimization problem tractable.

5.1.3 Results

We report the testing results of different approaches with the basic setting introduced above. We defer the testing results in extensive simulations with various settings, to show the priority and robustness of our algorithm, to the E-Companion EC.4, and the training results under all settings to the E-Companion EC.5.

Figure 6Testing results with the basic setting

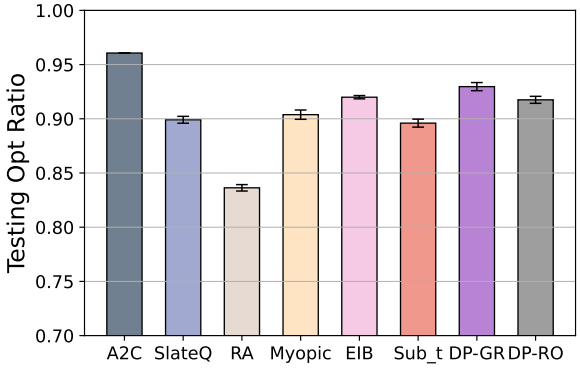


Figure 6 shows the comparison of all approaches while testing with the basic setting. The height of each cube indicates the mean of the 10 recorded testing optimal ratios against the offline upper bound resulting from each approach, and the deviation of the 10 tests is represented by the black line segment on the top of each cube. The first observation from the result is that our algorithm outperforms all benchmarks,

achieving an optimal ratio greater than 0.95. Among the benchmarks, the mean performance of the Random policy is 0.84, which is the worst as expected since it doesn't learn any information from the historical data set. The result shows that the Myopic policy and the Sub_t policy achieve an optimal ratio of 0.9, and the EIB policy, the DP-GR policy, and the DP-RO policy achieve an optimal ratio of 0.92. This shows that the policies that protect inventory for later customers indeed help achieve higher testing performance. While the result shows the prospect of adopting DRL for online assortment customization, SlateQ, another DRL-based method, performs worse than benchmarks other than the Random policy. We own the best performance of our A2C algorithm to two key factors. First, our algorithm interacts with the fitted Gated-Assort-Net simulator, which fits the generation of training transaction data better than the MNL choice model and thus also fits the underlying ground truth better. Second, the training of our A2C algorithm incorporates the training sequences, which can't be captured by the benchmarks. Interacting with the past arrival sequences helps Our A2C agent to learn an approximate optimal assortment policy for long-term revenue maximization. Learning from past arrival sequences also decreases the deviation of testing results in our algorithm, as can be seen from the testing result, because the learned A2C agent generates assortments flexibly according to the arrival information.

5.2 Real World Data Set Experiment

Here we conduct simulations based on a real-world dataset: Expedia[1], which is a data set about hotel transactions, consisting of the customer search sessions, the recommended assortments, the attributes of products in the assortments, and the choices of these sessions. We choose this data set from a practical point of view that the total number of rooms is fixed in a time horizon. We extract a stream of customer arrival sequences and the transaction record of each arriving customer from this data set and manually set the initial inventory level of each hotel to construct the environment. Since the ground truth choice mechanism generating this transaction data is unknown, we adopt Gated-Assort-Net to fit this data and act as feedback of the environment, which has been certified to fit the underlying ranking-based model well. We train and test our algorithm and benchmarks based on the well-fitted gated-Assort-Net. We explain the data preprocessing and simulator fitting process in E-Companion EC.6.

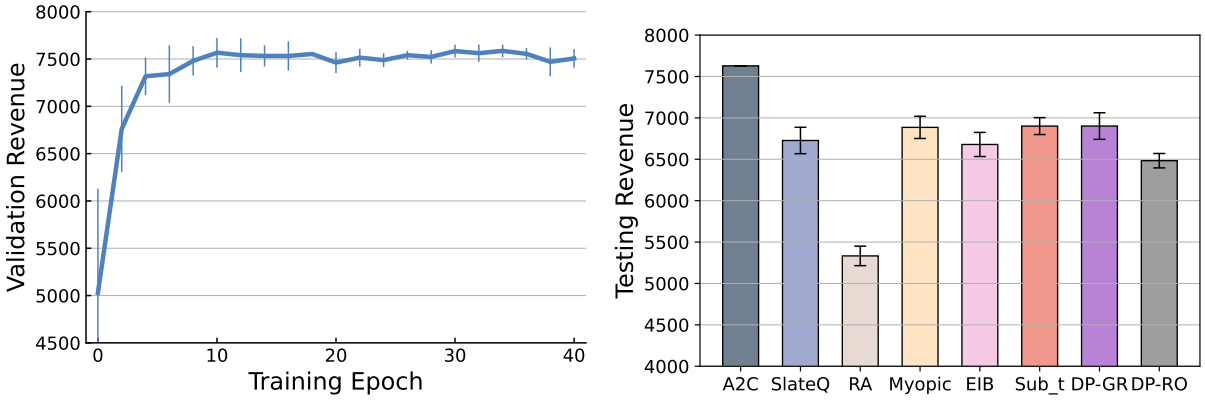
5.2.1 Performance of A2C Algorithm

We separate the customer sequences of the first 160 days for training our A2C agent and use the remaining 51 customer sequences for testing. We manually set the initial inventory level of each hotel as 2. The structure of our model is set the same as the model in the previous section. We train our model for 20 epochs and each epoch corresponds to the whole 160 training customer sequences. After every training epoch, we validate the current parameters of our model. The training process is triggered 5 times independently. The

[1] `https://www.kaggle.com/competitions/expedia-hotel-recommendations/overview`

training of Myopic and EIB based on the Gated-Assort-Net simulator requires transaction data from this simulator. We record the transaction data generated from one above 40-epoch A2C training process and use this data to fit the MNL parameters for implementing Myopic and EIB. Here we adopt the feature-based MNL model, which sets the utility of each product as a linear function of the product features, and we fit the coefficient of this linear function. After the A2C agent and the MNL parameters are trained, we test four approaches 10 times independently using testing customer sequences, considering the stochastic choice behavior of underlying customers.

Figure 7Training curve and testing result



We report the training process and testing performance in Figure 7. The training revenue of the A2C agent increases markedly from 5000 to 7500 in the first 10 epochs and remains stable in the later 30 training epochs. The deviation of 5 runs decreases from training epoch 5 to training epoch 40, which demonstrates that our algorithm converges to a relatively fixed point. From the testing result, we can see clearly that our method achieves the highest revenue with more than 7500, and the lowest testing deviation under the saved model parameter. The consistency of the training and testing results demonstrates the ability of our A2C approach to generalize to the unseen arrival sequences. The existence of more products means a larger action space and it's harder to produce an optimal policy. Our algorithm handles this case well and possesses greater advantages over benchmarks. On the other hand, the large gap between our A2C approach and the benchmarks shows that the benchmarks that rely on the MNL choice model can lose a large percentage of optimal revenue because of the misspecification of the underlying choice model, although the benchmarks may have theoretical guarantees assuming the MNL choice model.

6 Extension to Reusable Products

In this section, we divert our attention to a more general problem to consider reusable products. In Section 6.1, we describe how to modify the MDP and our model for the new problem. In Section 6.2, We further verify the performance of the modified model by conducting extended numerical experiments of Section 5.1.

6.1 Model Modification

We consider reusable products in this section. Note that if we set the usage time long enough, this problem is exactly the main problem we analyzed in the previous Section 3. In this point of view, considering reusable products in the online assortment problem leads to a more general case. Unlike the problem described in previous sections that the inventory level of each product decreases permanently, reusable products will come back into the inventory after some period of time since they are purchased by coming customers. Under this scenario, the assortment actions by the platform should care about not only the remaining inventory level of each product but also the products that are currently in use. If we use a state vector to record the rental time of each inventory unit, the number of possible states will grow exponentially with the number of products N and the length of the selling horizon T, which makes the dynamic program under stochastic arrival order and known usage time distributions intractable (Rusmevichientong et al. 2020). Thus the incorporation of reusable time makes the online assortment problem harder to analyze, and an approximately optimal policy is harder to produce.

In order to capture the products that are currently in use in this problem, we make a modification to our MDP formulation in Section 4.1. At the beginning of each time period, we receive the returned products before we make our assortment action. So the change in inventory levels depends not only on the current assortment action but also on historical sales in this new problem. The current assortment action influences the purchasing outcome and thus the decrease of inventory levels at this time period, and the overall historical sales influence the return-back incident of in-use products and thus the increase of inventory levels at this time period. Specifically, we use $\mathbf{o}_{t,z} \in \{0,1\}^N, z \in \mathcal{Z}$, which is a vector where the index of 1 indicates the purchased product, to represent the purchasing outcome at time period t by customer type z. Note that an all-0 vector $\mathbf{o}_{t,z}$ indicates that either $z \in \mathcal{Z}$ is not the customer type that arrived at t or customer type z arrived at t but did not purchase anything. We can infer the usage time information of each sold product at time period t based on the observed sales outcomes $\mathbf{O}_{t,z} = \{\mathbf{o}_{1,z}, \ldots, \mathbf{o}_{t-1,z}\}$. $\mathbf{O}_{t,z}$ updates to $\mathbf{O}_{t+1,z}$ in the end of time period t after the arriving customer makes his choice. Recall that the state of MDP we defined in Section 3 is $\mathbf{S}_t = (\mathbf{I_t}, z_t, t)$. Actually, the transition of $\mathbf{I_t}$ depends on the newly defined historical sales outcomes $\mathbf{O}_{t,z}, z \in \mathcal{Z}$. As a result, we add $\mathbf{O}_t = \{\mathbf{O}_{t,z}, z \in \mathcal{Z}\}$ to the state $\mathbf{S}_t$ at time period t to guide our assortment action, which forms a new MDP. In this way, the environment dynamics depend on customer arrival orders, their choice behaviors and the usage time of the rental products. However, there is a problem in the new MDP that the size of historical sales outcomes $\mathbf{O}_t$ grows over time. Oroojlooyjadid et al. (2022) face a similar problem to ours, and we follow them to record only the last L periods of sales outcomes $\mathbf{O}_t^L = \{\mathbf{O}_{t,z}^L, z \in \mathcal{Z}\}, \mathbf{O}_{t,z}^L = \{\mathbf{o}_{t-L,z}, \ldots, \mathbf{o}_{t-1,z}\}$ and use it to replace $\mathbf{O}_t$. Thus the state of the new MDP becomes

$$\mathbf{S}_t = (\mathbf{I_t}, z_t, t, \mathbf{O}_t^L),$$

We then describe how we deal with the historical sales outcomes $\mathbf{O}_t^L$. These sales outcomes are sparse, and we transform them by applying a set of time-series filters (Gabel and Timoshenko 2022). We apply different time-series filters for different products and different customer types. Each time-series filter is represented by a L-dimensional vector $\mathbf{w}_i^z \in \mathbb{R}^L, i \in \mathcal{N}, z \in \mathcal{Z}$ and a leaky rectified linear unit (ReLU) activation function $\sigma(\cdot)$ (Xu et al. 2015):

$$\sigma(x) = \begin{cases} x & \text{for } x \geq 0, \\ 0.2x & \text{for } x < 0. \end{cases}$$

For sales outcomes $\mathbf{O}_t^L = \{\mathbf{O}_{t,z}^L, z \in \mathcal{Z}\}$, the transformation can be written as

$$\mathbf{O}_t^N = \frac{1}{|\mathcal{Z}|} \sum_{z \in \mathcal{Z}} \sigma((\mathbf{O}_{t,z}^L)^T \odot [\mathbf{w}_1^z, \ldots, \mathbf{w}_N^z]^T),$$

where $\odot$ denotes the element-wise product. The result $\mathbf{O}_t^N \in \mathbb{R}^N$ reveals the rental information of each product, which is instructive for our assortment actions. We use $\mathbf{O}_t^N$ as an input of our network together with the current inventory level vector $\mathbf{I_t}$, the customer type indicator z_t, and t. The parameters in the time-series filters are updated together with other parameters in our model while training.

Different from existing literature on online assortment customization for reusable products where the usage time distributions are known, Our approach is flexible without assuming any form of usage time distribution. We regard the return of products as a part of the stochastic environment dynamics which ought to be learned through interaction with the environment.

6.2 Simulation Results

We conduct additional numerical experiments, following the basic setting introduced in Section 5.1. The modification of the underlying environment is that the particular product purchased by a certain customer type will be returned after a random time of usage.

Here the LP upper bound 5 is adapted to:

$$\begin{aligned} \max & \sum_{t=1}^{T} \sum_{a \in \mathcal{A}} \sum_{i=1}^{n} r_i p_{z_t}(i,a) y^t(a) \\ \text{s.t. } & \textit{inventory constraint} \\ & \sum_{a \in \mathscr{A}} y^t(a) = 1, \quad 1 \leq t \leq T, \\ & y^t(a) \geq 0, \quad 1 \leq t \leq T, a \in \mathcal{A}. \end{aligned}$$

Where the *inventory constraint* depends on the usage time distribution of the sold products. We set the usage time to follow a negative binomial distribution, which benefits the DP-GR policy and the DP-RO policy since it's the same distribution as the assumption in Rusmevichientong et al. (2020). To test the performance and robustness of our approach, we consider three cases of usage time distribution:

Case 1: the usage time of product i bought by customer type z follows a negative binomial distribution with parameter (s_i, η_i). Specifically, we set $s_i = 2, \forall i \in \mathcal{N}$, and (η_i) as 10 values evenly lie between 0.2 and 0.4. The *inventory constraint* in this case is expressed as:

$$\sum_{\tau=1}^{t} \sum_{a \in \mathscr{A}} (1 - F_i(t-\tau))\, p_{z_\tau}(i,a) y^\tau(a) \le c_i \quad 1 \le i \le n, 1 \le t \le T$$

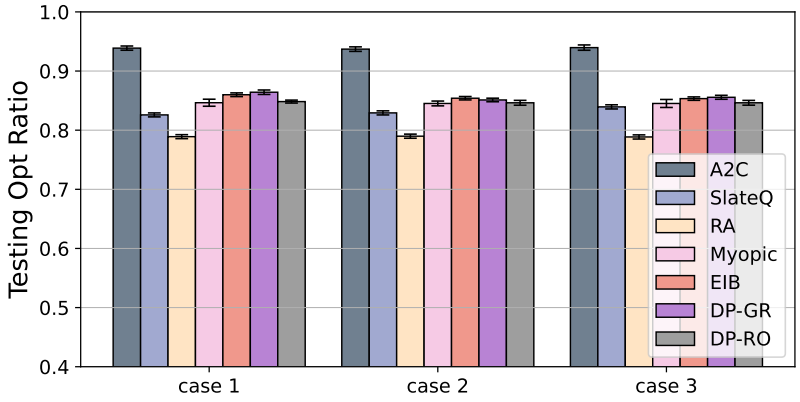
Case 2: the usage time of product i bought by customer type z follows a negative binomial distribution with parameter (s_z, η_z). Specifically, we set $s_i = 2, \forall i \in \mathcal{N}$, and $(\eta_1, \ldots, \eta_4) = (0.2, 0.25, 0.3, 0.35)$. The *inventory constraint* in this case is expressed as:

$$\sum_{\tau=1}^{t} \sum_{a \in \mathscr{A}} (1 - F_{z_\tau}(t-\tau))\, p_{z_\tau}(i,a) y^\tau(a) \le c_i \quad 1 \le i \le n, 1 \le t \le T$$

Case 3: the usage time of product i bought by the type of customer z follows a negative binomial distribution with parameter $(2, 0.25)$. Since all products follow the same usage time distribution, the *inventory constraint* in this case is expressed as:

$$\sum_{\tau=1}^{t} \sum_{a \in \mathscr{A}} (1 - F(t-\tau))\, p_{z_\tau}(i,a) y^\tau(a) \le c_i \quad 1 \le i \le n, 1 \le t \le T$$

Figure 8Training curve and testing result



We conduct five training processes and ten testing processes in each case. The testing results of three cases are shown in Figure 8, and the training results are shown in E-Companion EC.5. We test all the benchmarks except the Sub_t policy here for comparison.

In the testing results, we can see that although the benchmarks except the Random policy can achieve optimal ratios higher than 0.9 in the nonreusable case, here none of the benchmarks achieves an optimal ratio higher than 0.85 in all three cases, which shows that considering reusability further complicates the online assortment problem. However, our algorithm achieves nearly 93 % of the optimal revenue, outperforming benchmarks in all cases. This shows that the reusability nature in the environment dynamics has been learned through interaction and our algorithm can learn to offer near-optimal assortments in this harder case.

7 Conclusion and Future Work

We consider an online assortment customization problem with inventory and cardinality constraints. Unlike existing works that formulate this problem based on a specific or a class of choice models, this work is the first to propose a data-driven way based on reinforcement learning to learn a customized assortment policy. This success is due to a specific deep neural network model and the application of an A2C training algorithm. Through the interaction with a simulated environment built from historical data, our model learns an approximately optimal policy. Specifically, the interaction means trialing assortment actions to customers with their types recorded orderly in the historical data and observing the purchasing outcome from the feedback mechanism fitted from historical transition data. Furthermore, we adapt our model to the setting with reusable products. The rental time information at a certain time period is considered by adding purchasing history to the state, and a set of time series filters are adopted to transform them into the input of our model.

Our work is a new application of the DRL method in revenue management. Through the numerical experiments, we have verified that DRL with our well-designed model architecture can generate a customized assortment policy with high performance. In particular, we made the following observations from offline simulations:

- Under the environment with a realistic ranking-based ground truth choice model, the reinforcement learning framework produces a well-learned assortment policy through interaction with the simulator.

- Our A2C framework is more flexible than the Q-learning-based method SlateQ. The simulator in the training process of A2C can be in any form and should fit the existing transaction data as precisely as possible.

- The flexibility of A2C also leads to strong performance. Compared with the benchmarks, the policy learned by A2C achieves the highest long-term revenue when interacting with the customer sequences with ground truth feedback in the test data. The expected revenue loss is less than 95% compared with the offline optimal value, demonstrating that our model generalizes well and performs near optimal.

- The performance of our approach is robust in various settings under the synthetic environment and the environment based on a real-world data set. We also extend to consider reusable products. Our approach performs best under various usage time distributions.

We admit that one notable limitation of this work is the dependency on historical data for simulator construction, and the model's effectiveness and adaptability are reliant on the quality and representativeness of the historical data. The relationship between the performance of our approach and the amount and quality of historical data is a future direction to be investigated. This work also points to several other future directions.

First, we can consider the assortment problem in a more complex setting. In the online platform, the behavior of the choice depends on the position of the product and the assortment Abeliuk et al. (2016).

Moreover, the customer can click on multiple pages to search for fit products due to space constraints (e.g. Fata et al. 2019, Feldman and Segev 2022). Incorporating these effects into our framework is a promising direction.

The second direction is the combination of pricing strategy. Pricing is another mid-season trick to increase future revenue. Existing literature focuses on solving this problem under the MNL model (Lei et al. 2022). It will be interesting to solve a joint assortment and pricing problem in our framework.

Finally, future work can investigate how to consider the appearance of new products and the disappearance of existing products. Transfer learning is a possible tool for this problem, just as in Oroojlooyjadid et al. (2022).

References

Abeliuk, Andrés, Gerardo Berbeglia, Manuel Cebrian, Pascal Van Hentenryck. 2016. Assortment optimization under a multinomial logit model with position bias and social influence. *4OR*, 14 (1), 57-75.

Afsar, M Mehdi, Trafford Crump, Behrouz Far. 2022. Reinforcement learning based recommender systems: A survey. *ACM Computing Surveys*, 55 (7), 1-38.

Alomrani, Mohammad Ali, Reza Moravej, Elias B Khalil. 2021. Deep policies for online bipartite matching: A reinforcement learning approach. *arXiv preprint arXiv:2109.10380*, .

Aouad, Ali, Antoine Désir. 2022. Representing random utility choice models with neural networks. *arXiv preprint arXiv:2207.12877*, .

Aouad, Ali, Jacob Feldman, Danny Segev. 2022. The exponomial choice model for assortment optimization: an alternative to the mnl model? *Management Science*, .

Bellman, Richard. 1954. The theory of dynamic programming. *Bulletin of the American Mathematical Society*, 60 (6), 503-515.

Bello, Irwan, Hieu Pham, Quoc V Le, Mohammad Norouzi, Samy Bengio. 2016. Neural combinatorial optimization with reinforcement learning. *arXiv preprint arXiv:1611.09940*, .

Bentz, Yves, Dwight Merunka. 2000. Neural networks and the multinomial logit for brand choice modelling: a hybrid approach. *Journal of Forecasting*, 19 (3), 177-200.

Bernstein, Fernando, A Gürhan Kök, Lei Xie. 2015. Dynamic assortment customization with limited inventories. *Manufacturing & Service Operations Management*, 17 (4), 538-553.

Blanchet, Jose, Guillermo Gallego, Vineet Goyal. 2016. A markov chain approximation to choice modeling. *Operations Research*, 64 (4), 886-905.

Cai, Zhongze, Hanzhao Wang, Kalyan Talluri, Xiaocheng Li. 2022. Deep learning for choice modeling. *arXiv preprint arXiv:2208.09325*, .

Chen, Ningyuan, Ming Hu. 2023. Frontiers in service science: Data-driven revenue management: The interplay of data, model, and decisions. *Service Science*, 15 (2), 79-91.

Chen, Xi, Will Ma, David Simchi-Levi, Linwei Xin. 2020. Assortment planning for recommendations at checkout under inventory constraints. *Available at SSRN 2853093*, .

Chen, Yi-Chun, Velibor V Mišić. 2022. Decision forest: A nonparametric approach to modeling irrational choice. *Management Science*, .

Choi, Tsan-Ming, Subodha Kumar, Xiaohang Yue, Hau-Ling Chan. 2022. Disruptive technologies and operations management in the industry 4.0 era and beyond. *Production and Operations Management*, 31 (1), 9-31.

Delarue, Arthur, Ross Anderson, Christian Tjandraatmadja. 2020. Reinforcement learning with combinatorial actions: An application to vehicle routing. *Advances in Neural Information Processing Systems*, 33 609-620.

Deng, Yiting, Yuexing Li, Jing-Sheng Jeannette Song. 2022. A unified parsimonious model for structural demand estimation accounting for stockout and substitution. *Available at SSRN*, .

Farias, Vivek, Srikanth Jagabathula, Devavrat Shah. 2009. A data-driven approach to modeling choice. *Advances in Neural Information Processing Systems*, 22.

Fata, Elaheh, Will Ma, David Simchi-Levi. 2019. Multi-stage and multi-customer assortment optimization with inventory constraints. *arXiv preprint arXiv:1908.09808*, .

Feldman, Jacob, Danny Segev. 2022. The multinomial logit model with sequential offerings: Algorithmic frameworks for product recommendation displays. *Operations Research*, .

Feng, Yiding, Rad Niazadeh, Amin Saberi. 2019. Linear programming based online policies for real-time assortment of reusable resources. *Chicago Booth Research Paper*, (20-25),.

Gabel, Sebastian, Artem Timoshenko. 2022. Product choice with large assortments: A scalable deep-learning model. *Management Science*, 68 (3), 1808-1827.

Gallego, Guillermo, Anran Li, Van-Anh Truong, Xinshang Wang. 2015. Online resource allocation with customer choice. *arXiv preprint arXiv:1511.01837*, .

Gijsbrechts, Joren, Robert N Boute, Jan A Van Mieghem, Dennis Zhang. 2021. Can deep reinforcement learning improve inventory management? performance on dual sourcing, lost sales and multi-echelon problems. *Manufacturing & Service Operations Management*, .

Golrezaei, Negin, Hamid Nazerzadeh, Paat Rusmevichientong. 2014. Real-time optimization of personalized assortments. *Management Science*, 60 (6), 1532-1551.

Gong, Xiao-Yue, Vineet Goyal, Garud N Iyengar, David Simchi-Levi, Rajan Udwani, Shuangyu Wang. 2022. Online assortment optimization with reusable resources. *Management Science*, 68 (7), 4772-4785.

Goyal, Vineet, Garud Iyengar, Rajan Udwani. 2020. Asymptotically optimal competitive ratio for online allocation of reusable resources. *arXiv preprint arXiv:2002.02430*, .

Green, Etan A, E Barry Plunkett. 2022. The science of the deal: Optimal bargaining on ebay using deep reinforcement learning. *Proceedings of the 23rd ACM Conference on Economics and Computation*. 1-27.

Han, Yafei, Francisco Camara Pereira, Moshe Ben-Akiva, Christopher Zegras. 2022. A neural-embedded discrete choice model: Learning taste representation with strengthened interpretability. *Transportation Research Part B: Methodological*, 163 166-186.

Ie, Eugene, Vihan Jain, Jing Wang, Sanmit Narvekar, Ritesh Agarwal, Rui Wu, Heng-Tze Cheng, Tushar Chandra, Craig Boutilier. 2019. Slateq: a tractable decomposition for reinforcement learning with recommendation sets. *Proceedings of the 28th International Joint Conference on Artificial Intelligence*. 2592-2599.

Kalweit, Gabriel, Maria Huegle, Moritz Werling, Joschka Boedecker. 2020. Deep constrained q-learning. *arXiv preprint arXiv:2003.09398*, .

Karande, Chinmay, Aranyak Mehta, Pushkar Tripathi. 2011. Online bipartite matching with unknown distributions. *Proceedings of the forty-third annual ACM symposium on Theory of computing*. 587-596.

Kokkodis, Marios, Panagiotis G Ipeirotis. 2021. Demand-aware career path recommendations: A reinforcement learning approach. *Management Science*, 67 (7), 4362-4383.

Lei, Yanzhe, Stefanus Jasin, Joline Uichanco, Andrew Vakhutinsky. 2022. Joint product framing (display, ranking, pricing) and order fulfillment under the multinomial logit model for e-commerce retailers. *Manufacturing & Service Operations Management*, 24 (3), 1529-1546.

Li, Yuanzhi, Yang Yuan. 2017. Convergence analysis of two-layer neural networks with relu activation. *Advances in neural information processing systems*, 30.

Liu, Xiao. 2022. Dynamic coupon targeting using batch deep reinforcement learning: An application to livestream shopping. *Marketing Science*, .

McFadden, Daniel, et al. 1973. Conditional logit analysis of qualitative choice behavior, .

Mnih, Volodymyr, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. 2015. Human-level control through deep reinforcement learning. *nature*, 518 (7540), 529-533.

Oroojlooyjadid, Afshin, MohammadReza Nazari, Lawrence V Snyder, Martin Takáč. 2022. A deep q-network for the beer game: Deep reinforcement learning for inventory optimization. *Manufacturing & Service Operations Management*, 24 (1), 285-304.

Rana, Rupal, Fernando S Oliveira. 2014. Real-time dynamic pricing in a non-stationary environment using model-free reinforcement learning. *Omega*, 47 116-126.

Rusmevichientong, Paat, Zuo-Jun Max Shen, David B Shmoys. 2010. Dynamic assortment optimization with a multinomial logit choice model and capacity constraint. *Operations research*, 58 (6), 1666-1680.

Rusmevichientong, Paat, Mika Sumida, Huseyin Topaloglu. 2020. Dynamic assortment optimization for reusable products with random usage durations. *Management Science*, 66 (7), 2820-2844.

Sumida, Mika, Guillermo Gallego, Paat Rusmevichientong, Huseyin Topaloglu, James Davis. 2021. Revenue-utility tradeoff in assortment optimization under the multinomial logit model with totally unimodular constraints. *Management Science*, 67 (5), 2845-2869.

Sutton, Richard S, Andrew G Barto. 2018. *Reinforcement learning: An introduction*. MIT press.

Talluri, Kalyan T, Garrett Van Ryzin, Garrett Van Ryzin. 2004. *The theory and practice of revenue management*, vol. 1. Springer.

Watkins, Christopher JCH, Peter Dayan. 1992. Q-learning. *Machine learning*, 8 279-292.

Williams, Ronald J. 1992. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning*, 8 (3), 229-256.

Xie, Jiaohong, Yang Liu, Nan Chen. 2023. Two-sided deep reinforcement learning for dynamic mobility-on-demand management with mixed autonomy. *Transportation Science*, .

Xu, Bing, Naiyan Wang, Tianqi Chen, Mu Li. 2015. Empirical evaluation of rectified activations in convolutional network. *arXiv preprint arXiv:1505.00853*, .

Yang, Cenying, Yihao Feng, Andrew Whinston. 2022. Dynamic pricing and information disclosure for fresh produce: An artificial intelligence approach. *Production and Operations Management*, 31 (1), 155-171.

E-Companion

EC.1 Implementing details of benchmarks

-**Myopic-MNL**: Myopic-MNL is a greedy heuristic that does not take inventory issues into consideration. Using existing transaction data, we first fit the MNL parameter, where each product is endowed a utility u_i^z towards customer type $z \in Z$. For each assortment $a \subset \{1,\ldots,N\}$, the probability that type-z customer chooses product i is:

$$p_z(i,a) = \begin{cases} e^{u_i}/(1+\sum_{k\in a} e^{u_k}), & \text{if } i \in a, \\ 1/(1+\sum_{k\in a} e^{u_k}), & \text{if } i = 0, \\ 0, & \text{otherwise.} \end{cases} \tag{EC.1}$$

Upon arriving at a customer, Myopic-MNL solves a revenue maximization problem with cardinality constraint based on the MNL choice model:

$$a_t = \arg\max_{a\in A_t, |a|\le C} \sum_{i\in a} r_i p_{z_t}(i,a),$$

we adopt the StaticMNL algorithm proposed in Rusmevichientong et al. (2010) to solve this optimization problem.

-**EIB-MNL**: EIB-MNL is an online algorithm considering back-end supply chain constraints, with worst-case performance guarantees against the offline full-information case under MNL model (Golrezaei et al. 2014). The key idea is to compute a "discounted-revenue index" based on the products' remaining inventory. The revenue of products with low inventory levels is discounted, so we tend to hide it and show products with high inventory levels. The implementation of EIB-MNL also begins with the fitting of the MNL parameter. At each time period we solve the assortment optimization problem under cardinality constraint with the prices discounted by exponential penalty function $\Psi(x) = (e/(e-1))\,(1-e^{-x})$, where I_i^0 denotes the initial inventory level of product i and I_i^{t-1} denotes the inventory level of product i at the beginning of time period t:

$$a_t = \arg\max_{a\in A_t, |a|\le C} \sum_{i\in a} \Psi(I_i^{t-1}/I_i^0) r_i p_{z_t}(i,a).$$

-**Sub$_t$-MNL**: Sub$_t$-MNL (Bernstein et al. 2015) assumes the stochastic arrival pattern with a fixed selling length and a Poisson arrival process. That is, a customer arrives with probability λ each time period and the arriving customer belongs to segment $z \in \mathcal{Z}$ with probability ρ_z, with $\sum_{z\in\mathcal{Z}} \rho_z = 1$. Bernstein et al. (2015) only considers the case that all product prices are equal, and we examine its performance under the scenario where product prices are distinct. This leads to a dynamic program:

$$V_t(\mathbf{I}_t \mid z_t) = \max_{a_t \subset A_t} \left\{ \sum_{i\in a_t} \lambda p_{z_t}(i,a_t) \left(r_i - \Delta_{t+1}^i(\mathbf{I}_t)\right) \right\} + V_{t+1}(\mathbf{I}_t),$$

Sub_t approximates $\Delta^i_{t+1}(\mathbf{I}_t)$ with $\tilde{\Delta}^i_{t+1}(\mathbf{I}_t)$, by considering the substitution dynamics between products and the potential future demand for each product, to solve the intractable dynamic program as follows:

$$\max_{a_t \subset A_t} \left\{ \sum_{i \in a_t} \lambda p_{z_t}(i, a_t) \left(r_i - \tilde{\Delta}^i_{t+1}(\mathbf{I}_t) \right) \right\},$$

$$\tilde{\Delta}^i_{t+1}(\mathbf{I}_t) = r_i * \left[\Pr\left(D^{tS}_i > I_{ti} \right) - \sum_{j \neq i} \alpha^t_{ij} \Pr\left(D^{tS}_j \leq I_{tj}, D^t_i > I_{ti} \right) \right],$$

where

D^t_j is a Poisson distributed random variable with rate $d^t * (T-t)$,

$$d^t_j = \lambda \sum_{z \in \mathcal{Z}} \rho_z \frac{e^{u_{zj}}}{\sum_{k \in A_t} e^{u_{zk}} + 1},$$

$$D^{tS}_i = D^t_i + \sum_{j \in A_t \setminus \{i\}} \alpha^t_{ji} \left(D^t_j - I_{tj} \right)^+,$$

$$\alpha^t_{ij} = \sum_{z \in \mathcal{Z}} \rho_z \left(\frac{e^{u_{zj}}}{\sum_{k \in A_t \setminus \langle i \rangle} e^{u_{zk}} + 1} \right) \quad \text{for all } j \neq i.$$

Specifically, we fix the length T as the largest number of arrivals in the data set and estimate the arrival rate λ and customer type distribution $\rho_z, z \in \mathcal{Z}$ based on the historical arrival data. We also fit the MNL parameter from historical transaction data.

-**DP-GR**: DP-GR (Rusmevichientong et al. 2020) offers assortments based on the approximated value function of a dynamic-programming formulation. First, for a negative binomial usage time distribution with parameter $(2, \eta^z_i)$ for product i and customer type z, we set $u^z_{i,\ell} = \Pr\{\text{Duration}_i = \ell + 1 | \text{Duration}_i > \ell\} = \frac{f^z_i(\ell+1)}{\sum_{s=\ell+1}^{\infty} f^z_i(s)}$, where f^z_i is the probability mass function of the fitted distribution. Note that in the nonreusable case, $u^z_{i,\ell} = 0, \forall \ell = 0, 1, \ldots$. We set T as the maximum length in the training sequences, and fit the arrival probability of customer type $\rho_z, z \in \mathcal{Z}$ from the training sequences, $\sum_{z \in \mathcal{Z}} \rho_z = 1$. We then compute a set of approximated values $\hat{\nu}^{t,z}_{i,\ell}$ as follows:

• **Initialization**: Set $\hat{\theta}^{T+1}_i = 0$ and $\hat{\nu}^{T+1,z}_{i,\ell} = 0$ for all $i \in \mathcal{N}, z \in \mathcal{Z}, \ell \geq 1$.

• **Recursion**: For $t = T, T-1, \ldots, 1$, we compute $\hat{\theta}^t_i$ and $\hat{\nu}^{t,z}_{i,\ell}$ by using $\{\hat{\theta}^{t+1}_i : i \in \mathcal{N}\}$ and $\{\hat{\nu}^{t+1,z}_{i,\ell} : i \in \mathcal{N}, \hat{z} \in \mathcal{Z}, \ell \geq 1\}$ as follows. For each $z \in \mathcal{Z}$, let$\hat{A}^{t,z}$ be such that

$$\hat{A}^{t,z} = \arg\max_{|a| \leq C} \sum_{i \in \mathcal{N}} p_z(i, a) \left[r_i - \left(1 - u^z_{i,0} \right) \left(\hat{\theta}^{t+1}_i - \hat{\nu}^{t+1,z}_{i,1} \right) \right].$$

Once $\hat{A}^{t,z}$ is computed for all $z \in \mathcal{Z}$, for each $i \in \mathcal{N}$ and $z \in \mathcal{Z}$, let

$$\hat{\theta}^t_i = \hat{\theta}^{t+1}_i + \frac{1}{I_{t0}} \sum_{z \in \mathcal{Z}} \rho_z p_z \left(i, \hat{A}^{t,z} \right) \left[r_i - \left(1 - u^z_{i,0} \right) \left(\hat{\theta}^{t+1}_i - \hat{\nu}^{t+1,z}_{i,1} \right) \right]$$

$$\hat{\nu}^{t,z}_{i,\ell} = u^z_{i,\ell} \, \hat{\theta}^{t+1}_i + \left(1 - u^z_{i,\ell} \right) \hat{\nu}^{t+1,z}_{i,\ell+1} \quad \forall \ell = 1, 2, \ldots.$$

After the recursion, DP-GR offers the assortment a_t to arriving customer type $z \in \mathcal{Z}$ at time period t, based on the computed approximation values:

$$a_t = \arg\max_{|a| \leq C} \sum_{i=1}^{N} \mathbb{I}_{\{I_{ti} \geq 1\}} p_z(i, a) \left[r_i - \left(1 - u^z_{i,0} \right) \cdot \left(\hat{\nu}^{t+1,z}_{i,0} - \hat{\nu}^{t+1,z}_{i,1} \right) \right]$$

-**DP-RO**: DP-RO (Rusmevichientong et al. 2020) offers assortments based on the computed $\widehat{A}^{t,z}$ as in the DP-GR policy and the current state. Assuming that the usage time of product i follows a negative binomial distribution $\mathsf{NegBin}(s_i,\eta_i)$ where the η_i indicates a dissatisfying probability, they use $\boldsymbol{w}_i=(w_{i,0},\ldots,w_{i,s_i-1})$ to denote the current state of product i, where $w_{i,d}$ is the number of customers who are using product i and have been dissatisfied for d times, and use $\boldsymbol{F}_i(\boldsymbol{w}_i)=(F_{i,d}(\boldsymbol{w}_i):0\le d\le s_i-1)$ to denote the state at the next time period:

$$F_{i,d}(\boldsymbol{w}_i)=\begin{cases}\mathsf{Bin}(w_{i,0},\eta_i) & \text{if} \quad d=0,\\ \mathsf{Bin}(w_{i,d},\eta_i)+(w_{i,d-1}-\mathsf{Bin}(w_{i,d-1},\eta_i)) & \text{if} \quad d=1,\ldots,s_i-1,\end{cases}$$

Using the vectors $\boldsymbol{e}_0=(1,0,0,\ldots,0)\in\mathbb{R}^{s_i}$ and $\boldsymbol{e}_1=(0,1,0,\ldots,0)\in\mathbb{R}^{s_i}$, we can compute state-aware approximated values based on $\widehat{A}^{t,z}$ calculated in the DP-GR policy by using the recursion:

$$\begin{aligned}V_i^t(\boldsymbol{w}_i)=&\mathbb{E}\Big\{V_i^{t+1}(\boldsymbol{F}_i(\boldsymbol{w}_i))\Big\}+\\ &\mathbb{I}_{\left\{\sum_{d=0}^{s_i-1}w_{i,d}<I_{t0}\right\}}\rho_z p_z\left(i,\hat{A}^{t,z}\right)\cdot\quad\Big(r_i-\eta_i\,\mathrm{E}\Big\{V_i^{t+1}(\boldsymbol{F}_i(\boldsymbol{w}_i))-V_i^{t+1}(\boldsymbol{F}_i(\boldsymbol{w}_i)+\boldsymbol{e}_0)\Big\}\\ &\qquad\qquad-(1-\eta_i)\,\mathbb{E}\Big\{V_i^{t+1}(\boldsymbol{F}_i(\boldsymbol{w}_i))-V_i^{t+1}(\boldsymbol{F}_i(\boldsymbol{w}_i)+\boldsymbol{e}_1)\Big\}\Big)\end{aligned}$$

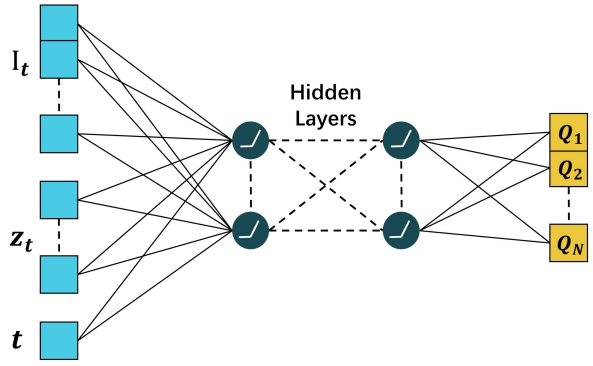
DP-RO offers the following assortment a_t to arriving customer type $z\in\mathcal{Z}$ at time period t

$$a_t=\arg\max_{|a|\le C}\sum_{i=1}^{N}\mathbb{I}_{\{I_{ti}\ge 1\}}p_z(i,a)\left[r_i-\left(1-u_{i,0}^z\right)\cdot\left(V_i^{t+1}\left(\boldsymbol{F}_i(\boldsymbol{w}_i)\right)-V_i^{t+1}\left(\boldsymbol{F}_i(\boldsymbol{w}_i)-\boldsymbol{e}_0+\boldsymbol{e}_1\right)\right)\right]$$

EC.2 Implementation details of SlateQ

The structure of the DQN in SlateQ is shown in Figure EC.1. We set the DQN as a two-hidden-layer fully-connected network with node sizes of 128 and 64. Each node is followed by a ReLU activation function. The input of the DQN is the state $\mathbf{S}_t=(\mathbf{I_t},z_t,t)$ of time t ($\mathbf{S}_t=(\mathbf{I_t},\mathbf{z}_t,t,\mathbf{O}_t^L)$ in the reusable case, and time filters introduced in Section 6.1 is also adopted for DQN). The output of the DQN is the Q-value vector of dimension N, where N is the number of products. We use the ε-Greedy, a replay pool and a target network in the training process as stated in Mnih et al. (2015). The initial value of ε is 1 and it decays linearly with a rate of 0.998 until it reaches 0.01. The training process starts when 1000 samples have been collected in the replay pool and we sample a batch of 8 samples to update the DQN parameters at each time t. The update process adhering to constraints is based on Algorithm 1 in Kalweit et al. (2020). When product i is sold out at time t, i.e. $I_{ti}=0$, we set the Q-value of product i as minus infinity, i.e. $Q_i=-\inf$, so that product i would not be selected into the assortment. After every training epoch, we validate the current parameters of DQN by traversing all the training sequences and recording the mean total revenue in these sequences. We save the model with the highest revenue for testing. An AdamOptimizer is used with the learning rate as 0.00001, and a linear decaying rate of learning rate is set as 0.99 every 100 steps.

Figure EC.1Structure of DQN



EC.3 Training and Testing

Our DRL approach learns through interactions with the environment. Since the underlying ground truth choice behavior is unknown in offline experiments, we fit a simulator based on the training transaction records generated from the ground truth. We adopt the Gated-Assort-Net (Cai et al. 2022) to act as the simulator, which possesses strong predictive power owing to the large capacity of the deep neural network. Since we don't incorporate any product or customer features, we fit 4 distinct feature-free Gated-Assort-Nets for 4 customer types. For customer type z, when faced with assortment $S_{z,t}$, feature-free Gated-Assort-Net with structure f and parameter θ_z predicts the choice probability vector over 11 products (remember that product-0 indicates the no-purchase option) as:

$$\mathbf{P}_{z,t} = f_{\theta_z}(S_{z,t}),$$

where $\mathbf{P}_{z,t} \in \mathbb{R}^{11}$ and $\sum_{i=1}^{11} P_{z,t}^i = 1$. The Gated-Assort-Net structure f is set as $(11, 11, 11)$, which means 11 input nodes, a one-hidden-layer network with node size 11 and ReLU activation function (Li and Yuan 2017), and 11 output nodes. Based on the recorded training transaction data, the parameter θ_z is trained by minimizing the CrossEntropy loss of the predictive probability vector $\mathbf{P}_{z,t}$ and an indicator vector of the actual outcome $\mathbf{Y}_{z,t} \in \mathbb{R}^{11}$:

$$-\sum_{i=1}^{11} Y_{z,t}^i \log\left(P_{z,t}^i\right),$$

We minimise this loss by adopting stochastic gradient descent.

After fitting the Gated-Assort-Net simulator, we train our A2C agent by Algorithm 1. We conduct one training epoch by traversing all 400 training customer sequences. Each training sequence simulates an arrival order of customer types from the environment for one horizon, and each arriving customer type reacts to our assortment action based on the fitted Gated-Assort-Net simulator. We set the shared layer as a three-hidden-layer network with node sizes of 64, 128, 64. Each node is followed by a ReLU activation function. The policy network is a one-layer Recurrent Neural Network (RNN) with 10 output nodes, and the value network is a fully connected linear layer with 1 output node. We update the parameters of our model

every 10 time periods based on the observed tuples, so the parameters are updated 10 steps for each training sequence with a mean length of 100. An Adam Optimizer is used with the learning rate of the shared layer, policy network and value network set as 0.0005, 0.0001, and 0.0001, respectively, and a linear decaying rate of learning rate is set as 0.999 every 100 steps. We train our model for 40 epochs in total. After every training epoch, we validate the current parameters of our model by traversing all the training sequences and recording the mean total revenue in these sequences. We ran the same training process 5 times to show that our algorithm converges to a stable point. We save the model with the highest validation revenue for testing.

Unlike our DRL approach, the SlateQ, Myopic-MNL, Sub_t-MNL, and EIB-MNL fit a set of MNL parameters from the training transaction data. We fit 4 distinct MNL models for 4 customer types, indicating 4 distinct choice behaviors. Since we do not incorporate product features, each vs corresponds to a deterministic utility vector $U_z = (u_{z,1}, \ldots, u_{z,10})$, $z = 1,2,3,4$. The fitting of $U_z, z = 1,2,3,4$ can be formulated as a Maximum Likelihood Estimation (MLE) problem:

$$\max_{U_z} \sum_{t=1}^{\tau_z} LL\left(S_{z,t}, y_{z,t}, U_z\right),$$

where $LL\left(S_{z,t}, y_{z,t}, U_z\right)$ is the log likelihood of observing transaction record $(S_{z,t}, y_{z,t})$ under MNL parameter U_z, calculated according to (EC.1). Note that Myopic-MNL, EIB-MNL, and Sub_t don't learn from the training sequences.

The training process of SlateQ is similar to A2C, except that each arriving customer type reacts to the assortment action based on the fitted MNL model. The parameters in DQN in SlateQ update according to Ie et al. (2019). Specifically, we deal with the inventory constraint and the cardinality constraint according to Algorithm 1 in Kalweit et al. (2020). The details of the structure of DQN and updating parameters are put in E-Companion EC.2.

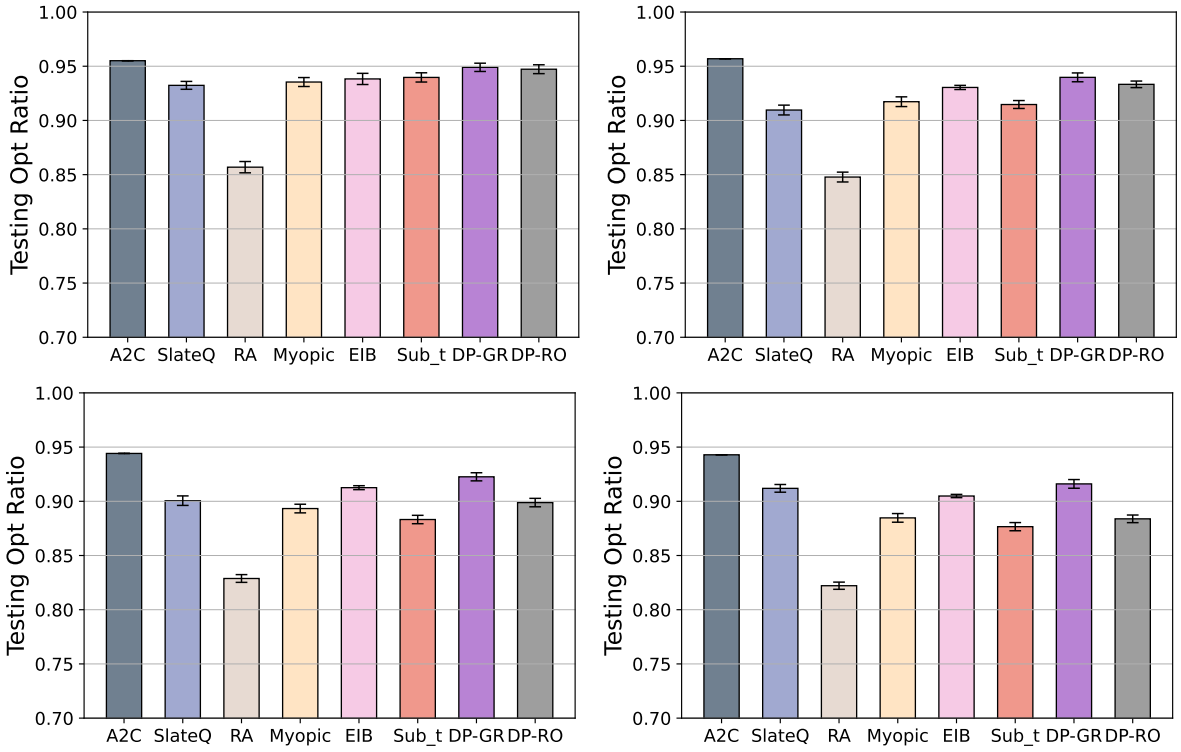
We test our approach and all the benchmarks with the testing customer sequences. Each arriving customer makes his choice decision towards an assortment according to the ground truth choice model. We adopt the mean revenue of all testing sequences to evaluate the four approaches. Since the choice behavior is stochastic, we conduct this test 10 times to reduce the randomness. This leads to 10 values of testing mean revenue recorded. The testing results are expressed as the fraction of the average LP upper bound, ranging from 0 to 1.

EC.4 Robustness Checks

In this section, we change the setting introduced in Section 5.1.3 and test the robustness of our method. First, we test the robustness of our model under different numbers of initial total inventories. We use the Load Factor (LF) to depict this change, which is defined as the ratio of the expected number of arriving customers in the selling season and the total inventory of all products, LF$= T / \sum_{i=1}^{10} I_{i0}$. The load factor we considered in the previous section is around $100/(10 \times 12) \approx 0.8$. A low LF means that the expected

number of customers is small and the total inventory is ample, while a high LF indicates that we may face a shortage for part of the products in the late periods. Since the length of each recorded sequence is fixed, we vary inventory scarcity levels by setting different initial inventory levels. We set the initial inventory level of each product as 10, 11, 13, and 14, respectively, which leads to various LFs as 1, 0.9, 0.75, and 0.7. The testing results of these four cases are reported in Figure EC.2. The training results are shown in E-Companion EC.5.

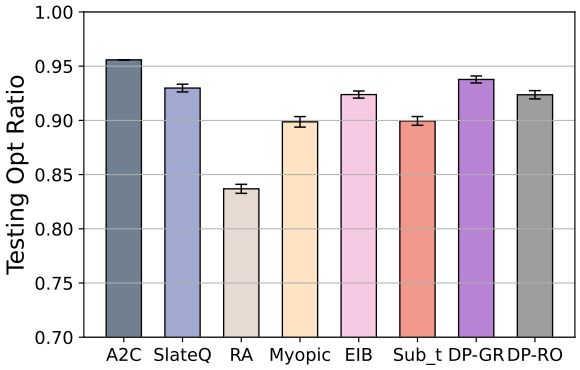
Figure EC.2Testing results with LF 1.0, 0.9, 0.75, and 0.7 (from left to right, up to down)



We can make several observations based on the results. Among all settings, A2C achieves optimal ratios of nearly 95%, which is better than all benchmarks, indicating DRL's power. This demonstrates that our A2C algorithm can learn an approximately optimal policy with different initial inventory levels. On the other hand, the Random policy performs worst among benchmarks and the DP-GR policy performs best among benchmarks. Another important observation is that when the LF decreases, that is, when we have more remaining products in the end, the differences between different policies become larger. Proper assortment planning is more essential with a lower LF since the performance of the Random policy becomes worse when LF decreases. The reason why the performance of different policies with LF 1.0 is indistinguishable is that nearly all products can be sold out in the end, and offer assortments randomly can achieve an optimal ratio higher than 0.85. The result further shows the priority of our method in settings with a low level of inventory scarcity.

We further test the robustness of our model under a different customer arrival pattern. We do this by changing κ, which is introduced in 5.1.1, to 0, so that different customer types arrive with equal probability at each time period. We set the initial inventory level of each product as 12 with LF$=0.8$. The testing results are shown in Figure EC.3. The comparison is quite similar to that in the basic case, indicating that our A2C approach can learn well under different customer arrival patterns.

Figure EC.3Testing results with $\kappa = 0$



Last, we test the robustness of our model under a larger case with 20 products and 6 customer types. We further test the scalability of our approach in EC.7 based on the real-world data set. We construct environments and generate synthetic data similar to the basic case, with the mean number of customers in one horizon being 120 and the initial inventory level of each product being 8 so that the LF$=0.75$. The testing results are shown in Figure EC.4. We can see that our A2C approach still performs the best among all policies, achieving the highest optimal ratio of 0.95. However, under this setting the Myopic policy and the Sub_t perform better than the EIB policy, the DP-GR policy, and the DP-RO policy. this result indicates that when the underlying ground truth choice model deviates from the MNL choice model, it's hard to tell which benchmark policy can perform the best under different settings. Actually, in the real world, it's hard to capture complex customer choice behavior accurately by a simple MNL choice model so the assortment decisions based on it are not optimal, while our A2C approach benefits from the current and future possible more powerful choice models.

EC.5 Training results of A2C

We report the training curves of the simulation under the basic setting with different initial inventory levels introduced in 5.1.3 and EC.4, and the reusable cases introduced in 6.2. The results are shown in the left and the right part of Figure EC.5, respectively.

In the training curve, the horizontal axis represents the number of training epochs and the vertical axis represents the revenue value. After each training epoch, we validate by traversing all 400 training sequences

Figure EC.4Testing results under a larger case

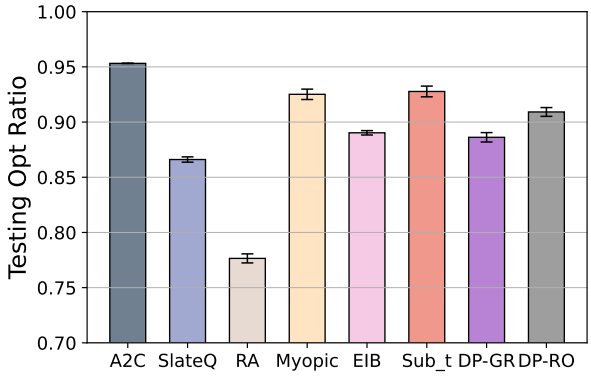
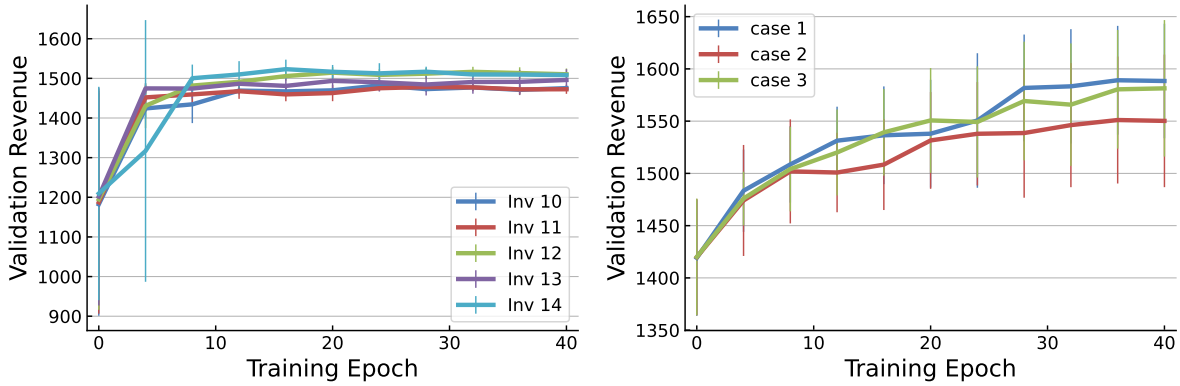


Figure EC.5Training curves (nonreusable case in the left, reusable case in the right)



with the current network parameters and recording the mean revenue of all horizons, with the adopted simulator as the feedback. The thick curves indicate the learning curves under different settings. The vertical line segments demonstrate the deviations of the data points of the 5 runs. In the nonreusable case, where "inv" indicates the initial inventory level of each product, We can see that the mean training revenue increases from 1200 to nearly 1500 in the first 10 epochs and remains stable in the last 30 training epochs, while the mean revenue is higher with a higher initial inventory level. The deviation of the mean training revenue is large in the first 10 epochs and decreases rapidly in the later epochs. In the reusable cases, the trends of training curves in different cases are similar, with all learning to relatively stable points.

EC.6 Expedia Data Pre-processing and Simulator Fitting

The Expedia data set provides the results of search queries for hotels on Expedia. The tabular data set includes customer information, hotel information, query information, and the result of each search. Each row records a recommended hotel in a certain search. The customer information columns are customer ID and location. The hotel information columns record the characteristics of a certain hotel, like its star, its location score and its price. The search information columns contain the input information by the customer before he clicks the search button, such as the search destination, and how many nights he wants to stay in the room. The customer can click, purchase or pass the recommended hotel, and we only care about

the purchase outcome here. We extract the rows with the same search destination ID 8192, which appears the most times in the data set. The extracted data set contains 126 hotels in total with this destination ID. We extract the 30 most popular hotels from them, which constitute 86.% transaction records of the total transaction records in this area. We further conduct two experiments with the 10 and 100 most popular hotels in this area and compare the results of these three cases to test the scalability of our approach.

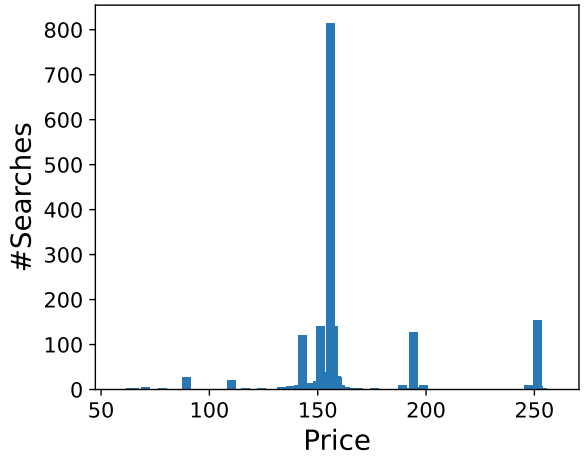
We pick 6 meaningful hotel feature columns of each hotel, which are shown in Table EC.1. The value of the feature vector of the same hotel may be different over time, so we set the feature values of each hotel as the mean of all records of this hotel. We normalize each feature using its mean and variance.

Table EC.1 Hotel Features

(1) prop starrating	The star rating of the hotel, from 1 to 5
(2) prop review score	The mean customer review score for the hotel on a scale out of 5
(3) prop brand bool	+1 if the hotel is part of a major hotel chain; 0 if it is an independent hotel
(4) prop location score1	A (first) score outlining the desirability of a hotel's location
(5) prop log historical price	The logarithm of the mean price of the hotel over the last trading period
(6) price usd	mean price of the hotel

We assign each search incident a customer type based on the power of consumption. Specifically, we classify all the search queries into 4 customer types according to the mean hotel price of all the recommended hotels in this search. Figure EC.6 shows the occurrences of different mean prices of search incidents. We can see that the mean price of most search queries lies around the value of 155. We separate the searches into 4 clusters according to the 0.25, 0.5, and 0.75 quantile values of these mean prices, which equal 151.3, 155.8 and 157.8. The number of searches in each cluster is 1367, 1664, 661 and 1230. This leads to an estimated distribution of customer types which equals $(0.278, 0.338, 0.134, 0.250)$.

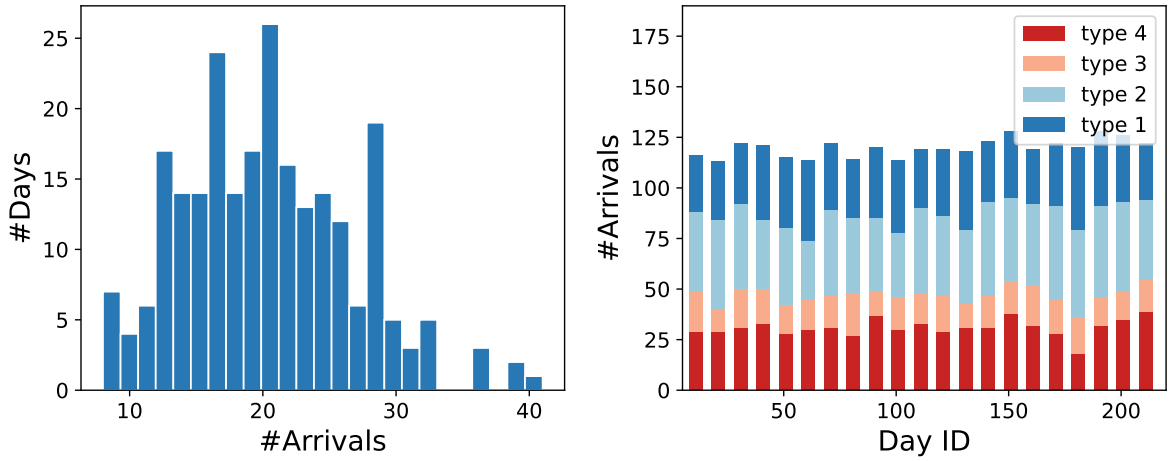
Figure EC.6Mean recommended price of search queries



After clustering underlying customers, we get the arriving order of customer types each day according to the search time recorded in the data set. We count the total number of customer arrivals each day in the

left Figure EC.7. We can see that the number of arrivals ranges from 6 to 41. We extract the days with more than 11 arrivals and less than 31 arrivals, which is the case for most days. Considering there are a number of customer arrivals unrecorded, we append the sequences of each day by sampling 100 customers according to the above-estimated distribution of customer types. The resulting customer component of each day is shown on the right of Figure EC.7.

Figure EC.7The arrival information of each day



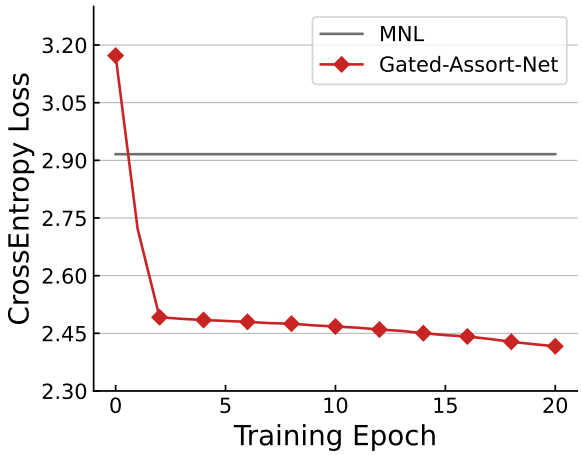
After the above data cleaning process, the statistics of the extracted data are shown in Table EC.2. We adopt a four-dimensional one-hot vector as the feature vector of each customer type, taking $(1,0,0,0)$ as an example of customer type 1. Each hotel is represented by a six-dimensional feature vector.

Table EC.2 Summary statistics of extracted data

# customer types	# hotels	# days	# search queries	# rows
4	30	211	4272	105158

From the result of the previous section, we get that the Gated-Assort-Net fit the underlying ground truth ranking-based model very well. While the ground truth choice model of the true environment is agnostic here, we fit a Gated-Assort-Net based on the extracted transaction data to act as the feedback of the true environment. Different from the training process of Gated-Assort-Net in the previous section, we consider customer features and product features here. We set the product encoder and customer encoder as fully connected layers $(6,10,8)$, and the Gated-Assort-Net is set as $(31,31,31)$. The training curve of this Gated-Assort-Net is shown in Figure EC.8. We can see that the well-trained Gated-Assort-Net achieves much lower CrossEntropy loss than the widely used MNL choice model. We regard the fitted Gated-Assort-Net as the ground truth feedback. We train and evaluate the DRL method and benchmarks based on it. Due to the large number of products and the complex structure of Gated-Assort-Net, we don't solve the aforementioned LP upper bound 5. We compare the exact resulting revenue of our method and all benchmarks.

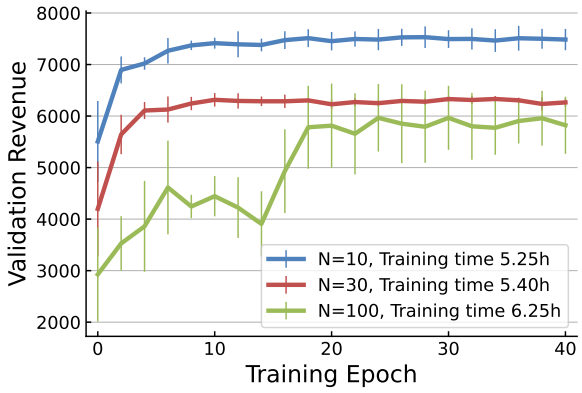
Figure EC.8Training curve of Gated-Assort-Net



EC.7 Scalability Test based on Real-world Dataset

In this section, we conduct extra experiments on different numbers of products based on real-world data to demonstrate the scalability of our approach. We keep the settings introduced in Section EC.6 all the same (including the number of customer types and the customer type arrival orders) except the number of products in the extracted data varying from 10, 30 to 100. With these three experiments, we compare the training curves, the training time, the testing results and the testing time of different approaches. We report the training curves and training time of these three cases in Figure EC.9. Note that there is no meaning in comparing the total revenue of these three cases since the number of products and their standardized prices are different, so we only care about the trends and the training time. We can see that all the DRL agents in three cases are trained to behave well after 20 epochs, and the training time of all three cases is below 7 hours. This demonstrates that the training cost of our approach is sustainable.

Figure EC.9Training curves with different numbers of products



We further report the testing results and the testing time in these three cases with different approaches. In the table, we report the percentage of testing revenue of each approach against the Random policy. By

testing time, we mean the total time which is used to propose an appropriate assortment for every customer of 50 arrival sequences, each with approximately 120 customers. We can see that the priority of our approach against the Random policy increases with the number of products. While other MNL-based policies like Myopic, EIB, Sub_t and DP-Greedy get worse when the number of products increases, probably because of the discrepancy between the fitted MNL choice model and the underlying Gated-Assort-Net choice model. As for the testing time, our approach scales well to 100 products with an increase of only 3 seconds. The underlying optimization problem behind the MNL-based methods is assortment optimization based on the MNL choice model with cardinality constraint, and we solve it based on the LP-based algorithm proposed in Sumida et al. (2021). We can see that with 100 products, the testing time of the MNL-based methods increase to more than 70 seconds. Among these approaches, Sub_t takes the longest time because of the complex approximation of $\tilde{\Delta}^i_{t+1}(\mathbf{I}_t)$. Both the testing result and the testing time demonstrate the scalability of our approach.

Table EC.3 Test results comparison with different numbers of products

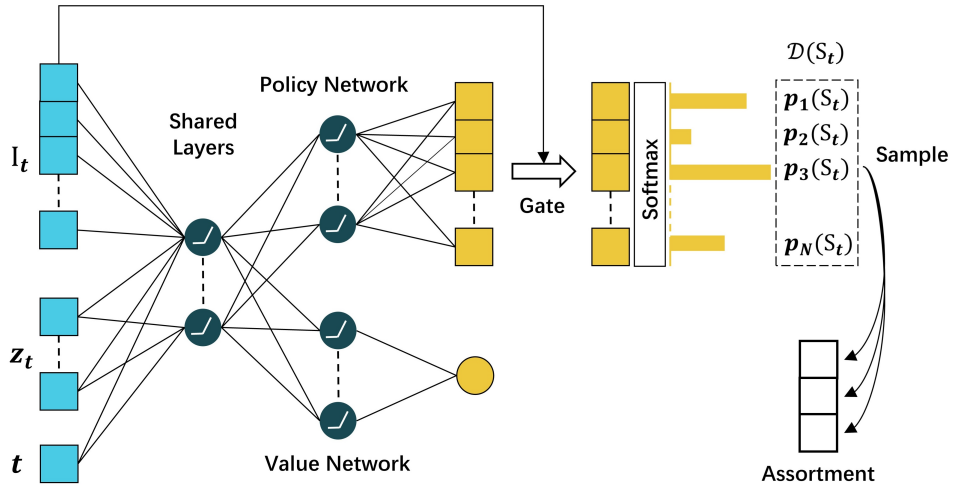
	N=10	N=30	N=100
A2C	133.4(12.0s)	143.0(12.1s)	202.8(15.0s)
Random	100(3.6s)	100(3.9s)	100(5.3s)
Myopic	113.0(16.8s)	129.1(34.1s)	88.0(75.3s)
EIB	107.1(16.7s)	125.2(34.0s)	68.3(72.1s)
Sub_t	113.6(109.0s)	129.4(886.0s)	86.4(7312.1s)
DP-Greedy	113.0(17.4s)	129.4(33.4s)	102.4(87.9s)

EC.8 Ablation Study

The key part of our network architecture is the RNN policy network. We investigate the help of this specific policy network through an ablation study which substitutes it for a standard multilayer perceptron (MLP) structure. We construct another network architecture with an MLP structure as the policy network for comparison. The detailed architecture is shown in Figure EC.10.

The only difference between this MLP-based model and our RNN-based model is the policy network. In our RNN-based model, a list of distributions is generated by the policy network sequentially, and an assortment is generated by sampling according to these distributions. However, in the MLP-based model, only one distribution is generated by the policy network, as shown in Figure EC.10 as $\mathcal{D}(S_t)$. A naive idea to generate an assortment adhering to assortment cardinality constraint C is to sample C times based on this distribution. There are two main drawbacks to this naive idea. First, the products in the assortment are sampled independently so it doesn't consider the cannibalization or complementary effects between different products. Our RNN-based model overcomes this problem through the passing of hidden states, so an optimal assortment is more likely to be found. Second, when we adopt the greedy policy to generate

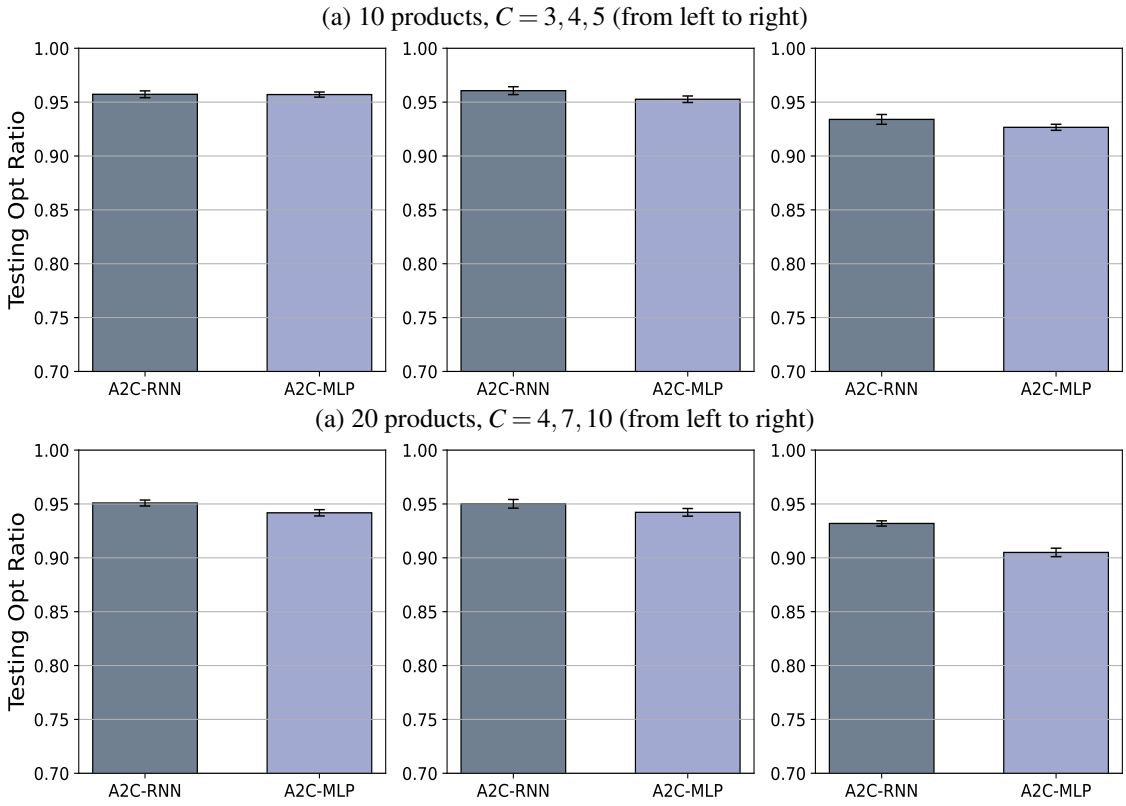
Figure EC.10Our DNN Architecture with MLP-based policy network



assortments, it can only generate assortments with size C, since no products can be chosen twice in the greedy manner. This means that it may miss the optimal assortment with a size smaller than C. Our RNN-based model overcomes this problem by considering an "eos" node to decide the end of the sequence when we generate an assortment sequentially. So our RNN-based model can generate assortments with size $\{1,\ldots,C\}$ when adopting the greedy policy in the testing process. We can note that the larger the assortment cardinality size is, the more severe are these two problems with the MLP-based model.

Through the comparison above it's clear that the assortment cardinality C is a key factor that determines the performance of the RNN-based model and the MLP-based model. Think about the extreme case that $C = 1$, the assortment-generation process in both the RNN-based model and the MLP-based model is just a single-step generation of distribution over all products so that the RNN-based model and the MLP-based model are indistinguishable. As a result, we compare the performance of these two models with varying assortment cardinality sizes. In the upper of Figure EC.11, we report the testing results of these two models under the basic setting, but with assortment cardinality sizes $C = 3, 4, 5$. We can see that with $C = 3$ the performance of these two models is nearly the same. As the cardinality size becomes larger, the RNN-based model performs better than the MLP-based model, while the performance of these two is still close. We further compare these two models under the setting of 20 products and 6 customer types as already introduced above, but with assortment cardinality sizes $C = 4, 7, 10$. From the results shown in the lower part of Figure EC.11, we can see that the priority of the RNN-based model becomes obvious with a larger number of products and a larger size of assortment cardinality constraint. It's also noticeable that the RNN-based model outperforms the MLP-based model under all settings. By this ablation study, we demonstrate that the transiting hidden states through the RNN layers help generate optimal assortments according to state observations.

Figure EC.11Ablation study with varying assortment capacity sizes

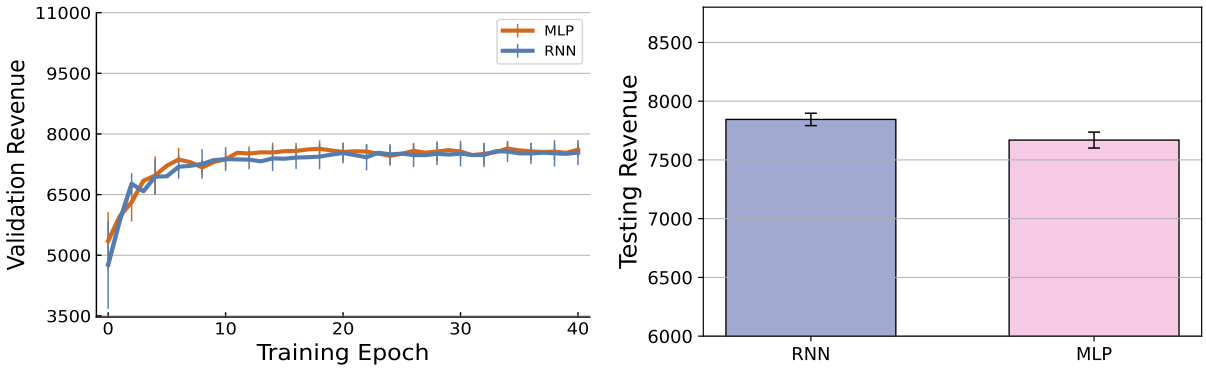


In addition, we verify our claims by comparing the performance of our RNN-based model with that of the MLP-based model in the context of Section 5.2. Here we start with the same setting in Section 5.2. That is, there are 30 products with the initial inventory level of each product as 2, and we set the cardinality size C as 4. The comparison of the two models is shown in part (a) of Figure EC.12. We can see that the difference is subtle here. The training curves of these two models are similar and the RNN-based model takes a small advantage over the MLP-based model in the testing results.

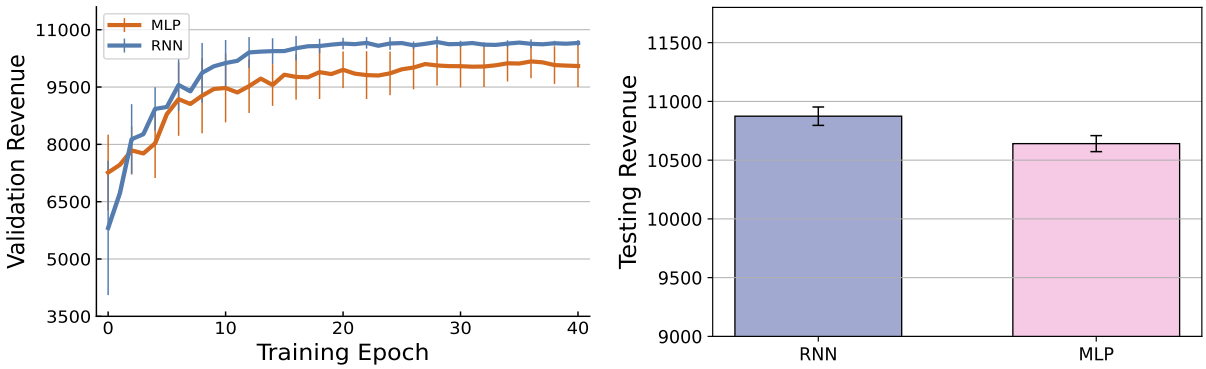
Next, we enlarge the cardinality size to 8 and 15. This means a larger action space and a harder task. We also enlarge the initial inventory of each product to 3 to avoid all the products being sold out under these more flexible cardinality settings. The comparison of the two models is shown in part (b) and part (c) of Figure EC.12. We can see that our RNN-based model takes bigger advantages under these two settings, both in the training process and in the testing results. Furthermore, when the cardinality size was enlarged from 8 to 15, the performance of our RNN-based model improves because of a more flexible constraint on the assortments, while the performance of the MLP-based model degenerates. This demonstrates that the MLP-based model performs badly with a large cardinality size. Our RNN-based model outperforms the MLP-based model by 1.7%, 2.6%, and 11.2% under three settings respectively. This result strongly supports that the RNN policy network is well-suited for the assortment-generation task.

Figure EC.12Comparison of RNN-based model and MLP-based model under three different settings

(a) $I_0 = 2, C = 4$



(b) $I_0 = 3, C = 8$



(c) $I_0 = 3, C = 15$